

**UNITED STATES PATENT AND TRADEMARK OFFICE**

---

**BEFORE THE PATENT TRIAL AND APPEAL BOARD**

---

MICROSOFT CORP.,  
Petitioner,

v.

ROYAL J. O'BRIEN,  
Patent Owner.

U.S. Patent No. 8,380,808  
Issued: Feb. 19, 2013  
Filed: Nov. 24, 2008

Patent Title: DYNAMIC MEDIUM CONTENT STREAMING SYSTEM

---

*Inter Partes* Review No. 2021-00015

---

**DECLARATION OF WILLIAM ANTHONY MASON**

**TABLE OF CONTENTS**

|      |                                                                                          |    |
|------|------------------------------------------------------------------------------------------|----|
| I.   | INTRODUCTION AND ENGAGEMENT .....                                                        | 1  |
| II.  | BACKGROUND AND QUALIFICATIONS .....                                                      | 1  |
| III. | MATERIALS CONSIDERED AND INFORMATION RELIED UPON<br>REGARDING '819 PATENT .....          | 7  |
| IV.  | LEGAL STANDARDS .....                                                                    | 10 |
| V.   | BACKGROUND .....                                                                         | 11 |
| A.   | State of The Pertinent Art .....                                                         | 11 |
| B.   | Overview of the '808 Patent .....                                                        | 20 |
| C.   | Claims Construction.....                                                                 | 20 |
| D.   | Person Having Ordinary Skill in the Art.....                                             | 30 |
| VI.  | THE MICROSOFT OFFERED PRIOR ART FAILS TO DEMONSTRATE THE<br>'808 PATENT IS OBVIOUS ..... | 30 |
| A.   | Summary of Microsoft's Prior Art References .....                                        | 30 |
| B.   | Analysis of the '808 Patent in Light of Microsoft's Prior Art References.....            | 31 |

I, William Anthony Mason, do hereby declare as follows:

**I. INTRODUCTION AND ENGAGEMENT**

1. I have been retained as an independent expert on behalf of Royal O'Brien ("O'Brien") in connection with the above-captioned Petition for *Inter Partes* Review ("IPR") to provide my analyses and opinions on certain technical issues related to U.S. Patent No. 8,308,808 (hereinafter "the '808 Patent").

2. I am being compensated at my usual and customary rate of \$500 per hour for the time I spent in connection with this IPR. My compensation is not affected by the outcome of this IPR.

3. Specifically, I have been asked to provide my opinions regarding whether claims 1-20 (each a "Challenged Claim" and collectively the "Challenged Claims") of the '808 Patent would have been obvious to a person having ordinary skill in the art ("POSITA") as of at least as early as November 24, 2008. It is my opinion that each Challenged Claim would not have been obvious to a POSITA after reviewing the prior art presented by the Petitioner.

**II. BACKGROUND AND QUALIFICATIONS**

4. I am an expert in file systems, file system filter drivers, including file system filter drivers that provide isolation, encryption, and compression as well as kernel mode and user mode software related to file systems and file system filter drivers, including on Microsoft's Windows Operating System. I have published articles, taught seminars, architected, designed, developed, reviewed, and debugged file systems, file system filter drivers, and kernel mode software and user mode software development related to file systems and file system filter drivers, including on Microsoft's Windows Operating System.

DECLARATION OF WILLIAM ANTHONY MASON

Case IPR2021-00015

U.S. Patent No. 8,380,808

5. I explain below at ¶¶53-54 the knowledge of a POSITA with respect to the '808 Patent; I have at least that level of knowledge, particularly in the area of Microsoft Windows file systems, file system filter drivers, and related technologies.

6. In formulating my opinions, I have relied upon my training, knowledge, and experience in the relevant art. A copy of my curriculum vitae is appended to this declaration as Appendix A and provides a description of my professional experience, including my academic and employment history, conference participation, teaching experience, and more. The following is a summary of my relevant qualifications and professional experience.

7. I received a Bachelor of Science in Mathematics, with a concentration in Computer Science, from the University of Chicago in 1987. I received a Master of Science in Computer Science from the Georgia Institute of Technology in 2017.

8. I am currently a full-time PhD student at the University of British Columbia; I started my PhD program in September 2017. I also work part-time as a technical consultant through my wholly owned company wamason.com LLC. My primary focus is on providing services related to operating systems, with an emphasis in storage and networking technologies.

9. Following my graduation from the University of Chicago, I worked for two years (1987-1989) with Professor David Cheriton at Stanford University in his Distributed Systems Group (DSG). I was responsible for developing various technologies, including diskless boot for workstations running the V operating system, developed by the DSG, ethernet drivers, a simple kernel debugger, and an experimental transport protocol (see IETF RFC 1045) for use in BSD 4.2 UNIX.

10. In summer 1989 I joined Transarc, a start-up company in Pittsburgh, PA, through my association with Dr. Jeff Eppinger, a faculty member at Stanford and one of the founders of

Transarc. From 1989 to September 1994, I worked for Transarc developing distributed file systems, including the Transarc AFS 3.0 product, and the DCE/DFS file system. My work on the DCE/DFS file system included developing the transactional journal mechanism that was part of the Episode physical file system. DCE/DFS has not been widely adopted, though many of the components of the DCE have been adopted, including the Episode file system, which remains in production use today. My CV references two of the papers that we published about our work in DCE/DFS.

11. From 1994 to 1995 I worked at FORE Systems, a Pittsburgh-based startup that revolutionized the field of asynchronous transfer mode (ATM) networking equipment. My responsibilities at FORE Systems revolved around developing support for ATM networking in Novell's Netware, Microsoft's Windows NT, and IBM's OS/2 operating systems, as well as representing FORE Systems' interests in local area network (LAN) emulation at the ATM Forum, an international standards committee focused on defining global standards for the ATM networking community.

12. From 1995 to 2016 I worked and was a minority owner at OSR Open Systems Resources, Inc. ("OSR"). My title was Consulting Partners, of which the company had two. OSR specialized in operating systems software architecture, design, and development. For most of the time that I was with OSR, I focused primarily on Windows file systems and related technologies. During my time at OSR, I developed and published the first publicly available file system filter driver example for Windows NT 3.5/3.51 on CompuServe, a commonly used computer bulletin board service of that era. Subsequently, I designed and developed a general-purpose framework for constructing file system filter drivers, as well as publishing several freely available articles regarding the issues and considerations for file system filter drivers and file systems. For example,

as part of my work at OSR, I designed and implemented one of the first file system filter drivers for an anti-virus product for McAfee.

13. I also worked extensively over my time with OSR with dozens of companies that licensed the OSR Filter Driver Development Kit (FDDK) and constructed unique products that utilized file system filter drivers for Windows. In addition, I was also responsible for designing and developing the OSR File Systems Development Kit, which OSR licensed to parties that wished to accelerate their implementation of local and distributed file systems for the Windows NT (and subsequent) platforms. As part of my duties at OSR, I actively participated with Microsoft's "Installable File System Plugfest", aiding and giving invited talks. In my role as Consulting Partner at OSR I was engaged to review the source code of a pre-release version of the new Microsoft filter manager. Filter manager is a legacy file system filter, the same as the OSR FDDK, that provided a rich interface for developing "file system minifilters". I conducted that review, identifies potential issues or concerns to Microsoft as part of a written report for them, and suggested approaches Microsoft might wish to consider in addressing and enhancing filter manager. Microsoft made changes to filter manager to address my concerns and implemented some of the improvements I proposed. Microsoft ultimately released the new minifilter environment as part of Service Pack 2 for Windows XP and has provided updated versions of that in subsequent service packs and versions of Windows. During my tenure at OSR, as part of my responsibilities, I also provided documentation to Microsoft about file systems development, particularly related to security. These documents were incorporated into the Microsoft Windows Installable file systems toolkit. Derivatives of that documentation are still part of the Microsoft Windows Driver Kit (WDK), which is used to develop file systems, and file system minifilters on the Windows platform.

14. In addition, from 1996 to 2016, through OSR I provided training in the development of Microsoft Windows device drivers, file system drivers, and file system minifilter drivers, as well as kernel debugging for Windows. I designed, developed, maintained, and presented all those courses during my tenure at OSR. These classes were offered in both public settings, in which attendees from numerous companies and organizations attended, as well as private settings, in which I would travel to the company's location and present the material to a restricted group. For example, I presented "Developing File Systems for Windows", "Developing File System Minifilters for Windows", and "Kernel Debugging for Windows" numerous times between 1996 and 2016 at Microsoft in various locations, including their Redmond, WA, Issaquah, WA and France facilities. I met Neal Christiansen (the lead inventor on application US 2006/0117018A1) prior to him joining Microsoft, when he attended my "Developing File Systems for Windows" course in 1996 and have worked with him over the years since. Similarly, I have worked with Ravi Pudipeddi (US 6993603), Molly Brown, Ravinder Thind, and Brian Dewey through their efforts in building the Microsoft Filter Manager, as well as Microsoft Plugfests, and my work with Windows file systems. I have worked with David Golds as well in the past, though prior to his involvement in Filter Manager. David Golds engaged me, through OSR, to present the "Developing File Systems for Windows" course to the file systems development teams at Microsoft almost two decades ago.

15. I have personally architected, designed, and developed numerous file systems, file system filter drivers, and file system minifilter drivers for Microsoft Windows, including several that have been distributed by Microsoft as part of their own product offerings. For example, Microsoft acquired Softricity in 2006; Softricity's application streaming platform, which became the basis of App-V, a streaming on-demand application virtualization for Microsoft Windows, used

the OSR File Systems Development Kit (FSDK) as part of its implementation and Microsoft continued to use it for many years after the acquisition. Similarly, Microsoft acquired a SQL server backup solution from UK-based Redgate Software. As part of that acquisition, Redgate engaged me, through OSR, to convert their legacy file system filter to a file system minifilter solution that implemented the specific functionality Redgate needed. Redgate's filter was an example of an "isolation filter" that I originally proposed and implemented as part of my responsibilities at OSR. A primary motivation for using isolation filters was to enable construction of a specific class of file system minifilters that control the presentation of data to applications in a way that is distinct from the actual data stored in the files. For example, we used the isolation filter model to implement data encryption filter and minifilter drivers, compression filter and minifilter drivers, as well as support for streaming on-demand delivery systems that supported arbitrary sized data units.

16. From 2007 through 2014, unrelated to my work with OSR, I worked with Royal O'Brien and Bitraider to resolve issues with the legacy file system filter driver that he had implemented. The Bitraider file system filter driver was an integral part of the streaming on-demand game delivery system that Bitraider was offering commercially. As an outgrowth of the original work on the Bitraider minifilter, I agreed to design and implement a file system minifilter that would utilize the "isolation filter" model that I continued to refine. The Bitraider streaming filter needed that level of control because of the novel way in which his product provided usage-aware delivery of the contents of files, unrelated to the underlying details of the storage.

17. Since 2015, I have been working as an Instructional Associate for the Georgia Institute of Technology, helping to deliver several of their graduate classes for their Online Master of Science in Computer Science program, including CS 6200 (Graduate Introduction to Operating

# DECLARATION OF WILLIAM ANTHONY MASON

Case IPR2021-00015

U.S. Patent No. 8,380,808

Systems), CS 6250 (Computer Networks), CS 6460 (Educational Technology), and CS 7210 (Distributed Systems). My responsibilities at Georgia Tech include helping students understand the concepts in the “systems” area of computer science, as well as mentoring graduate students in conducting research in improving the use of technology in education.

18. Since 2016, I have been working part-time for my wholly owned company, providing technical consulting, including litigation support services.

19. Since 2017, I have been a PhD student at the University of Computer Science, in Vancouver, British Columbia. My PhD supervisors (since October 2019) are Margo Seltzer (UBC), and Ada Gavrilovska (Georgia Tech). My research focus is in the area of file systems, including encryption, compression, naming, evaluation, and integration with storage technologies.

20. I have been awarded 11 United States patents, as more fully identified in my Curriculum Vitae.

### **III. MATERIALS CONSIDERED AND INFORMATION RELIED UPON REGARDING '819 PATENT**

21. In preparing this declaration, I have reviewed the following materials:

#### **MICROSOFT’S LIST OF EXHIBITS**

| <b>No.</b> | <b>Description</b>                                                                       |
|------------|------------------------------------------------------------------------------------------|
| 1001       | U.S. Patent No. 8,308,808 (“the ’808 Patent”)                                            |
| 1002       | File History of U.S. Patent Application No. 12/276,998 (808 Patent)                      |
| 1003       | (Not Used)                                                                               |
| 1004       | (Not Used)                                                                               |
| 1005       | U.S. Patent No. 6,574,618 to Eylon (“Eylon”)                                             |
| 1006       | U.S. Patent Application Publication No. US 2006/0117018 to Christiansen (“Christiansen”) |
| 1007       | U.S. Patent No. 6,993,603 to Pudipeddi (“Pudipeddi”)                                     |
| 1008       | U.S. Patent No. 7,496,565 to Thind et al. (“Thind”)                                      |
| 1009       | Internet Archive Affidavit and Webpage, “File System Filter Drivers” (Microsoft 2004)    |
| 1010       | FilterDriverDeveloperGuide                                                               |

## DECLARATION OF WILLIAM ANTHONY MASON

Case IPR2021-00015

U.S. Patent No. 8,380,808

|      |                                                                                                                                                                                                                                                 |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1011 | Overview-May04                                                                                                                                                                                                                                  |
| 1012 | ContextManagement                                                                                                                                                                                                                               |
| 1013 | Filter_Manager                                                                                                                                                                                                                                  |
| 1014 | E. Zadok, "On Incremental File System Development," ACM Transactions on Storage, Vol. 2, No. 2, May 2006, Pages 161-196 ("Zadok").                                                                                                              |
| 1015 | U.S. Patent No. 6,311,221 to Raz ("Raz")                                                                                                                                                                                                        |
| 1016 | U.S. Patent No. 7,617,253 to Prahlad ("Prahlad")                                                                                                                                                                                                |
| 1017 | Webpage, <a href="https://docs.microsoft.com/en-us/windowshardware/drivers/kernel/windows-kernel-mode-i-o-manager">https://docs.microsoft.com/en-us/windowshardware/drivers/kernel/windows-kernel-mode-i-o-manager</a> (describing "I/O stack") |
| 1018 | Microsoft Computer Dictionary (defining "routine," "object," "handle," "open")                                                                                                                                                                  |
| 1019 | Microsoft Computer Dictionary (defining "routine," "object," "handle," "open")                                                                                                                                                                  |
| 1020 | U.S. Patent Application Publication No. 2008/0075285 to Poli et al. ("Poli")                                                                                                                                                                    |
| 1021 | Webpage, <a href="https://docs.microsoft.com/enus/dotnet/api/system.io.fileinfo.length?view=netcore-3.1">https://docs.microsoft.com/enus/dotnet/api/system.io.fileinfo.length?view=netcore-3.1</a> (describing "size"/" length")                |
| 1022 | Webpage, <a href="https://www.cs.fsu.edu/~myers/c++/notes/fileio.html">https://www.cs.fsu.edu/~myers/c++/notes/fileio.html</a> (describing "file stream")                                                                                       |
| 1023 | Webpage, <a href="https://en.wikipedia.org/wiki/C%2B%2B">https://en.wikipedia.org/wiki/C%2B%2B</a> (describing "file stream")                                                                                                                   |
| 1024 | Webpage, <a href="https://en.wikipedia.org/wiki/Streaming_media">https://en.wikipedia.org/wiki/Streaming_media</a>                                                                                                                              |
| 1025 | Webpage, <a href="https://en.wikipedia.org/wiki/Hierarchical_storage_management">https://en.wikipedia.org/wiki/Hierarchical_storage_management</a>                                                                                              |
| 1026 | Unused                                                                                                                                                                                                                                          |
| 1027 | Unused                                                                                                                                                                                                                                          |
| 1028 | U.S. Patent No. 7,441,096 to Kitamura ("Kitamura")                                                                                                                                                                                              |
| 1029 | Smith, "Cache Memories," 14 Computing Surveys No. 3 (1982)                                                                                                                                                                                      |
| 1030 | U.S. Patent No. 6,732,183 to Graham ("Graham")                                                                                                                                                                                                  |
| 1031 | Jacob, Memory Systems (Elsevier 2008)                                                                                                                                                                                                           |
| 1032 | Infringement Contentions, Exhibit B to Amended Complaint, <i>O'Brien v. Microsoft Corp.</i> , No. 2:19-cv-01625-RAJ (Sept. 11, 2020), ECF No. 24                                                                                                |

## MY LIST OF EXHIBITS

| No.  | Description                                          |
|------|------------------------------------------------------|
| 2001 | File-System Development with Stackable Layers (1994) |
| 2002 | The Episode File System (1992)                       |
| 2003 | The DEcorum Architecture (1990)                      |

## DECLARATION OF WILLIAM ANTHONY MASON

Case IPR2021-00015

U.S. Patent No. 8,380,808

|      |                                                                                                                                                                                                                                                                                          |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2004 | Windows XP Lifecycle<br>( <a href="https://docs.microsoft.com/en-us/lifecycle/products/windows-xp">https://docs.microsoft.com/en-us/lifecycle/products/windows-xp</a> )                                                                                                                  |
| 2005 | Wikipedia: History of Computer Games (1948-Present)<br>( <a href="https://en.wikipedia.org/wiki/History_of_video_games">https://en.wikipedia.org/wiki/History_of_video_games</a> )                                                                                                       |
| 2006 | Wikipedia: Maze War<br>( <a href="https://en.wikipedia.org/wiki/Maze_War">https://en.wikipedia.org/wiki/Maze_War</a> )                                                                                                                                                                   |
| 2007 | IETF RFC 1045<br>( <a href="https://tools.ietf.org/pdf/rfc1045.pdf">https://tools.ietf.org/pdf/rfc1045.pdf</a> )                                                                                                                                                                         |
| 2008 | The Game Asset Pipeline (Ben Carter, 2004); pp 269-276, 287-295                                                                                                                                                                                                                          |
| 2009 | Tools of the Trade: What Do I Need to Create Interactive Games?<br>(December 1, 1997)<br>( <a href="https://www.awn.com/animationworld/tools-trade-what-do-i-need-create-interactive-games">https://www.awn.com/animationworld/tools-trade-what-do-i-need-create-interactive-games</a> ) |
| 2010 | Wikipedia: Internet Access<br>( <a href="https://en.wikipedia.org/wiki/Internet_access">https://en.wikipedia.org/wiki/Internet_access</a> )                                                                                                                                              |
| 2011 | IETF RFC 1945<br>( <a href="https://tools.ietf.org/pdf/rfc1945.pdf">https://tools.ietf.org/pdf/rfc1945.pdf</a> )                                                                                                                                                                         |
| 2012 | Synchronization and Caching Issues in the Andrew File System, CMU ITC Tech Report 63<br>( <a href="http://reports-archive.adm.cs.cmu.edu/anon/anon/home/anon/itc/CMU-ITC-063.pdf">http://reports-archive.adm.cs.cmu.edu/anon/anon/home/anon/itc/CMU-ITC-063.pdf</a> )                    |
| 2013 | Pricing the Internet (MacKie-Mason and Varian, 1993)                                                                                                                                                                                                                                     |
| 2014 | Web Brownout (Wired, 1996)                                                                                                                                                                                                                                                               |
| 2015 | The Impending Demise of the File System (Nielsen, 1996)                                                                                                                                                                                                                                  |
| 2016 | Transition to Advanced Format 4K Sector Hard Drives<br>( <a href="https://www.seagate.com/ca/en/tech-insights/advanced-format-4k-sector-hard-drives-master-ti/">https://www.seagate.com/ca/en/tech-insights/advanced-format-4k-sector-hard-drives-master-ti/</a> )                       |
| 2017 | Windows NT File System Internals: A Developer's Guide (1997, Nagar)                                                                                                                                                                                                                      |
| 2018 | Wikipedia: Information Theory<br>( <a href="https://en.wikipedia.org/wiki/Information_theory">https://en.wikipedia.org/wiki/Information_theory</a> )                                                                                                                                     |
| 2019 | 1994, RFC 1738 (URLs)                                                                                                                                                                                                                                                                    |
| 2020 | 2005, RFC 3986 (URIs)                                                                                                                                                                                                                                                                    |
| 2021 | Daley and Neumann, "A General-Purpose File System for Secondary Storage", AFIPS 1965 (Fall, Part 1), pp 203-229<br>( <a href="https://doi.org/10.1145/1463891.1463915">https://doi.org/10.1145/1463891.1463915</a> )                                                                     |
| 2022 | "Main memory caching of Web documents" Evangelos P. Markatos (1996)                                                                                                                                                                                                                      |
| 2023 | "Reducing WWW latency and bandwidth requirements by real-time distillation", Armando Fox and Eric Brewer (1996)                                                                                                                                                                          |
| 2024 | Wikipedia – File System (2021)                                                                                                                                                                                                                                                           |
| 2025 | A Fast File System for UNIX (1984)                                                                                                                                                                                                                                                       |
| 2026 | The Cedar File System (1988)                                                                                                                                                                                                                                                             |
| 2027 | NOVA: A Log-structured File System for Hybrid Volatile/Non-volatile Main Memories (2016)                                                                                                                                                                                                 |
| 2028 | Wikipedia – Database (2021)                                                                                                                                                                                                                                                              |

DECLARATION OF WILLIAM ANTHONY MASON

Case IPR2021-00015

U.S. Patent No. 8,380,808

|      |                                                                                                                   |
|------|-------------------------------------------------------------------------------------------------------------------|
| 2029 | Microsoft Research Tech Report MSR-TR-2011-109 (also published at CSCW '13 pp 1125-1136)                          |
| 2030 | MIT Lecture on Computer and Network Security including CBC                                                        |
| 2031 | Porting the Coda File Systems to Windows                                                                          |
| 2032 | FIPS 197                                                                                                          |
| 2033 | Dynamic Storage Allocation in the Atlas Computer, Including an Automatic Use of a Backing Store (1961)            |
| 2034 | Bhati US 7,191,189B2                                                                                              |
| 2035 | Microsoft and Intel Work Together to Significantly Decrease Application Start-Up Time for Customers of Windows 98 |
| 2036 | Wikipedia: Myst                                                                                                   |
| 2037 | OSR FDDK Version 3.0 Data Sheet                                                                                   |
| 2038 | Supported DCE and DFS Version                                                                                     |
| 2039 | One level storage system                                                                                          |
| 2040 | Organization and Retrieval of Records Generated in a Large-Scale Engineering Project                              |
| 2041 | On Computable Numbers with an Application to the Entscheidungsproblem                                             |
| 2042 | Replication in the Harp File System (1991)                                                                        |

**IV. LEGAL STANDARDS**

22. This section provides my understanding of the applicable legal principles regarding the standards for patentability, which I have used in forming my opinions regarding the '808 Patent.

23. I understand that Microsoft has the burden to prove that the claims challenged in its IPR petitions are not patentable by a preponderance of the evidence. I understand “preponderance of evidence” to be just enough evidence to make it more likely than not that its position is correct.

24. I understand that patentability of each claim challenged in the petitions is assessed based on documents that predate the '808 Patent, such as other patents and printed publications, which are referred to as “prior art.”

25. It is my understanding that a claim is not patentable if it is directed to subject matter that is not new or if the subject matter recited in the claim would be obvious to a person of ordinary

skill in the art in the field of the invention at the time of the invention. I understand that an obviousness analysis involves a consideration of (1) the scope and content of the prior art; (2) the differences between the claimed invention and the prior art; (3) the level of ordinary skill in the pertinent field; and (4) secondary considerations of non-obviousness. I understand that secondary considerations of non-obviousness include failure of others, copying, unexpectedly superior results, perception in the industry, commercial success, and long-felt but unmet need. It is also my understanding that obviousness can be established by combining or modifying the teachings of the prior art to produce the claimed invention where there is some teaching, suggestion, or motivation to do so. Additionally, it is my understanding that obviousness can be established when the claimed invention is predictable based on the prior art.

26. I understand that obviousness is determined based on the point of view of a person of ordinary skill in the art. I also understand that the person of ordinary skill in the art has ordinary creativity.

27. I understand that, in an IPR proceeding, the claims must be given their broadest reasonable interpretation in view of the specification. After interpreting the claims in this manner, the claims can then be compared to the prior art.

## **V. BACKGROUND**

### **A. State of The Pertinent Art**

28. I understand that the date of invention of the '808 Patent is its filing date of November 24, 2008. I will describe the perceived need for and state of the art of dynamic streaming content delivery preceding the filing date.

29. I begin this section with explanations of basic concepts in computer games, file systems, and Windows file system filter and minifilter drivers, which may clarify the descriptions

of the '808 Patent and prior art references that follow, as well as the descriptions of the state of the pertinent art below. While some of the concepts that I describe were well known in the art of file systems, file system filters, and file system minifilters, their combination with game technologies ran counter to their ordinary use; the combination of these taught in the '808 patent were a novel new combination from these ordinarily unrelated fields.

30. The term “file” is one that has meaning when used in the context of computer systems that does not entirely match its ordinary English language meaning. While there are similarities to the ordinary meaning of this term, there are sufficient differences that it can lead to misunderstandings. Microsoft themselves have pointed out the distinction between the ordinary English language meaning of the term “file” from how it is used in the computing field in their technical report “What is a File” in 2011:

“It would seem perfectly reasonable for the man or woman on the street to assume that the term file, when used in connection to a PC, reflects quite closely what they mean when they use the same term to describe the things they have in their non-digital world. One might be digital and the other real, but they are the same sort of things: file. Unfortunately, the relationship is not as simple as this – and indeed to think it might be or that there is even any straightforward analogue between the use of this word in everyday life and with regard to computing is quite egregious. It turns out the word covers many things; a number of things in ordinary life and a rather different set of things when used in reference to computers. All told so many things, in fact, that the way that it gets used to cross refer between these domains (and indeed sometimes within these domains) often leads to solecisms – to absurd meanings and mistaken understandings.” (Exhibit 2029).”

31. I have construed the term “file” to be an opaque logical container of digital data. It is *opaque* because the purpose of a file is to contain *arbitrary* data. It is *logical* because data within it follows some unknown organizational structure, despite its opacity. The contents do not dictate how the file is named, stored, or retrieved. The fact that the contents of a file are treated as opaque is important in distinguishing it from structured data, such as a record in a database. An application

that stores its data in a database declares, up front, to the database the structure of each individual record.

32. The term file system in the computing field is used to describe the software component that “controls how data is stored and retrieved.” (Exhibit 2024). Despite the simplicity of the concept, there are numerous file systems, each with radically different file systems, often linked to the storage mechanism used for the file system. For example, there are disk file systems (Exhibit 2025), distributed file systems (Exhibit 2026), and file systems for persistent memory (Exhibit 2027). One important design element of file systems is that, aside from distinguished information commonly called “meta-data” the data contents of a file are opaque to the file system. In other words, the file system does not utilize knowledge of the structure of the contents of the file. A file system is distinct from a database, in which the structure of the information is explicitly declared (Exhibit 2028). By utilizing knowledge of the structure of the data, a database can provide additional features to application programs that are not generally available from a file system.

33. File systems are a standard mechanism in operating systems, including the Windows operating system, for providing a fairly uniform set of services to applications that allow naming, storing, and retrieving the contents of the generic units of logical storage that we refer to as a “file”; we can aggregate a set of files into a collection that is traditionally referred to as a “directory” though in some fields the term “folder” is often used as an historical nod to the original motivation for the file system model in the 1950s, which sought to replicate the physical files, folders, and cabinets of a traditional paper filing system (Exhibit 2040). The file system model has proven to be powerful and even while file systems are generally a topic only briefly touched upon in academic classes on operating systems, file systems have been and continue to be a subject of innovation and exploration for more than a half-century. One common model for file systems is

to couple them with a persistent storage device, such as a hard disk drive, solid state disk (SSD), or optical storage (CD/DVD). All of these devices expose a similar interface in which data is read and written in fixed-sized units that are typically referred to as “sectors” or “blocks”. Thus, to update a single byte in such a block-structured system requires that the file system read the entire block into memory, update the single byte that needs to be changed, and then write the entire block back to storage. The concept of block structured storage is so ingrained that the introduction of solid-state disks included a layer that maps the random access memory of the underlying media (e.g., flash memory) to the block structured interface of a hard disk drive. One motivation for solid state disks to implement a block structured interface is that media file systems have been optimized to work with the sector/block model.

34. File system filter drivers are a technique in which the behavior of an existing file system is monitored, controlled, and/or augmented. The idea of layering functionality on an existing file system is an idea that was formally described by John Heidemann and Gerald Popek in 1994 (Exhibit 2001). I was personally involved in constructing layered file systems as early as 1990, as described in our papers (Exhibits 2102 and 2013). The source code for the system we created before 1996 and demonstrating how we constructed stackable file systems in several UNIX platforms is still publicly available. One such copy is available in my Github.com account: (<https://github.com/fsgeek/dce122>). Windows NT 3.1 included support for file system filter drivers at its initial release in 1993. I began writing and distributing public samples of file system filters by 1994 and my company, OSR Open Systems Resources, Inc. began licensing a commercial version of my work in that time period. I note that a file system filter driver only works when it is filtering an existing file system. It cannot “stand alone” or function without a file system.

35. The popularity of file system filter drivers created issues and challenges on Windows, which inspired Microsoft to develop and deploy a new model of file system filters. Microsoft's new file system filter model is known as "file system minifilters". The minifilter model was introduced to provide better support for file system filter drivers on the Windows platform, primarily to improve interoperability and decrease abnormal terminations of the Windows operating system (a "crash" or "blue screen of death"). File system minifilters were first introduced in Windows as of Windows XP Service Pack 2 (SP2) in September 2004; subsequent service packs were released to make it available in Windows 2000 (Exhibit 2004). The functionality of a file system minifilter requires that it be combined with a legacy file system filter driver, known as "filter manager". When combined, these two components are functionally equivalent to a functional legacy file system filter driver as described in ¶33. Microsoft spent many years encouraging developers to convert their existing ("legacy") file system filter drivers to use the new minifilter driver model because use of the minifilter driver model improved interoperability between third party components that included a file system filter driver.

36. Computer games have existed nearly as long as computers. Graphical computer games have consistently pushed innovation in the computing field by demanding more of hardware and operating systems. I helped add an efficient network protocol, which utilized a special form of the internet protocol (IP) known as IP-Multicast, to permit efficient multi-player capabilities to the X11 port of Maze War (Exhibit 2006) in 1988 as part of my work in implementing IETF RFC 1045 (Exhibit 2007). By using IP-Multicast, it was possible for multiple people to play an interactive combat game over an internet with minimal lag due to more efficient use of the network. Performance of games, and their reliance on networking is a theme that continues to be vital to the field of computer gaming today.

37. Windows has been and continues to be a popular gaming platform. The Windows operating system has been and continues to be used by Microsoft as part of their XBOX gaming consoles. In my experience, the computer game field and the file system filter driver field are not tightly related; it is rare to find someone that has worked in both fields. In none of the cases that I personally know would I describe those individuals as POSITAs; each case that I recall have been individuals with extra-ordinary creativity. While computer games use file system services, computer games do that through a standard interface used by essentially all applications for accessing files that provides primitives like open, close, read, and write. Computer games have thus developed independently from the evolution of file systems, file system filters, and file system minifilters. For example, a common term of art in computer games is the “asset”, which refers to a discrete element used in a computer game to render some aspect of the game. Examples of game assets include, but are not limited to textures, bitmaps, audio clips, video clips, etc. These individual elements can be the smallest possible addressable unit on the given computer system, typically one byte, but can also be as large as necessary – a deliberately vague size description. These individual elements (“assets” or “data packs”) can be packed into a one or a small number of files, stored in distinct and separate files, or even be stored and retrieved from a database or other structured storage instead of a file system. The term “asset” is one that is one familiar to someone of ordinary skill in the art of computer games (Exhibit 2009). Games often consist of thousands of distinct assets; the assets are not stored as individual files but instead are typically combined into one or more files that the game program uses when rendering the game as part of interacting with the human player. The idea that files within a file system have structure is not a new one. In 1962 it was known in the context of file systems and its impact on performance (Exhibit 2039).

38. In general, playing computer games is a leisure activity outside the business of developing such computer games. As such, computer games are not generally of interest to businesses (other than businesses involved in developing the games). Thus, it is unlikely that someone focused on the computing needs of business would ordinarily be involved in, or even interested in, the computer game field and its unique requirements.

39. Part of what drives the difference between business applications and games is the distinction between business applications and graphical content. For example, one might install Adobe Photoshop onto their computer system. The actual code for an application in the late 1990s was less than 50MB. However, the images that I create and manipulate with Adobe Photoshop could be 1MB each. Thus, even a few hundred photos could easily dwarf the size of the program.

40. Thus, business applications are (relatively) small. Business applications are then used to create content that is, in aggregate, larger. As we increase the media richness of that content such as embedded video, animation, visual effects, audio, captioning, etc. the size of the corresponding files also increases.

41. Games exacerbate the challenges of size because games not only provide static media content, but the components necessary to generate dynamic media content on the fly. Thus, games included potentially thousands of different graphical elements, some large, many small. These elements are used together to dynamically generate 3D contents. While there is some similarity between streaming a business application across a local area network, or even a high-speed wide area network, there would have been no need for a streaming on-demand application delivery system to consider the needs of the gaming community – or any media rich content delivery – in the late 1990s.

42. However, the home game market started to change in the early 2000s. The home game market changes were driven by the dramatic improvements the networking field was able to bring to residential homes. For example, one key change was rapid growth in performance, including significant increases in bandwidth and decreases in latency available to residential network users – the birth of broadband networking. Bandwidth is a measure of how much data can be transferred per unit time, such as the number of “megabits per second” (Mb/s), which is often used today to describe the bandwidth of commercial internet services. In the time of Eylon (Exhibit 1005) home users were primarily using dialup services capable of speeds up to 56Kb/s (kilobits per second) (Exhibit 2023). To transfer a 1GB (giga-byte) file using dialup services would require more than 36 hours of time. Thus, games for download were small. Larger games were distributed via CD-ROM, which had a capacity of up to 720MB.

43. For example, the game *Myst*, which I played on the PC platform in 1994, was in its time a large game that was distributed on CD-ROM; indeed, the adoption of CD-ROM on computers is partially attributed to *Myst* itself (Exhibit 2036). It was impractical to consider performing streaming download of such a game, even four years later (the time of Eylon).

44. In the late 1990s, business class external networks were typically much faster than what was available to casual home users. At the same time, the critical links used to connect the Internet were approximately the same as a good home broadband service, such as 45Mb/s, which is the speed offered by a dedicated T3 connection from the telephone company and was used for the Internet backbone in the time of Eylon. The costs of obtaining high speed internet connections (1.544Mb/s and up) was prohibitive for home users.

45. A business intranet in the 1990s, such as Microsoft’s own “Corpnet”, would have offered substantially higher speed data transfers that were available to residential internet users.

Large companies had data centers that contained server class machines with high-speed connections from individual computers. Thus, streaming business applications were possible while streaming game delivery was not practical. The environment of fast direct wired high bandwidth corporate networks and slow dial-up low bandwidth residential networks is the environment that gave rise to the teachings of Eylon.

46. Streaming content delivery is not a new idea; it was certainly not a new idea at the time of Eylon (Exhibit 1005). The Transmission Control Protocol (TCP), which makes up one of the key network communications protocols that form the backbone of the Internet, was designed to allow streaming data between two computers over the Internet. One common UNIX utility, “ftp” used the File Transfer Protocol (FTP) over TCP/IP to stream a file from one computer to another computer. The Hyper Text Transfer Protocol (HTTP) was adopted in May 1996 by the Internet Engineering Task Force (Exhibit 2011) and describes the streaming data transfer protocol that is still used by Internet browsers and servers to stream data over the Internet.

47. Data Caching is not a new idea; it was certainly not a new idea at the time of Eylon. The Andrew File System, to which I contributed to its commercial version (“AFS 3.0”) at Transarc in 1989, specifically provided data caching with strong coherence guarantees (Exhibit 2012). The Andrew File System, and AFS 3.0 both worked by streaming data from a file server to an individual computer where that data was being used in fixed size units referred to as “chunks”. When data changed on the original file server, a client that had cached it would be notified of that change so the client could update their caches accordingly. AFS was a complete file system and was available on the Windows platform prior to the time of Eylon.

48. Cache coherence, updating, notification, and the other concepts involved in managing local and remote modifications to a file over a network is not a new idea; it was certainly

not a new idea at the time of Eylon and thus not of Christiansen, either, since the two systems are solving quite similar problems. The DCE/DFS file system project, which was the successor to AFS 3.0 (described in ¶46), provided strong cache coherency guarantees and permitted simultaneous updates to a single file from multiple clients, which distinguished it from prior work where cache coherency was done just on an individual file basis. DCE/DFS was a complete file system and was available on the Windows platform (Exhibit 2038). The Coda project, a spin-off of the Andrew File System project at Carnegie-Mellon University also provided distributed caching, with the additional consideration of disconnected operation; I was involved in providing support for the Coda port to Windows (Exhibit 2031).

**B. Overview of the '808 Patent**

49. The '808 Patent discloses methods, devices, and systems for streaming applications to a local computer that combines standard concepts from the file systems community, including caching, write-back, data validation, and streaming data download with the concept of an asset from the game community, realized on the Windows platform using a file system minifilter to augment the capabilities of the Windows file systems to dynamically acquire data needed by the application, both predictively as well as “on demand” should the predictive mechanism not properly predict those needs.

**C. Claims Construction**

50. I understand that claims in an IPR are to be given their broadest reasonable interpretation in view of the specification.

51. The term “**streaming on demand system**” should be construed to mean a system that consists of a data that is maintained on one or more electronic data storage systems, and is provided to one or more computers, distinct from the data storage systems for use and processing.

DECLARATION OF WILLIAM ANTHONY MASON

Case IPR2021-00015

U.S. Patent No. 8,380,808

For example, a web browser is one embodiment of a “streaming on demand system” because when given a valid uniform resource identifier (URI), it can combine information into a visual display with which a human user can interact. For example, the name of a web page, such as <https://www.microsoft.com>, is an instance of a URI. The web browser will interpret the URI to be a reference to a set of computers available on the network. For example, when I used the nslookup tool on my Windows 10 desktop computer for converting domain names to IP addresses, I was told that [www.microsoft.com](http://www.microsoft.com) was available using three network addresses: 2600:1409:6800:197::356e, 2600:1409:6800:19a::356e, and 23.44.161.156. My web browser would then use one of those addresses to open a network connection, using port 443, which corresponds to the numeric value of the **https** port, which is specified by one portion of the URI. The web server would then stream a block of information to the web client. If that block of information is properly formatted hypertext markup language (HTML), which is the norm, my web browser will interpret the contents of that information. In my example case, the returned HTML document includes numerous references to other content, located on the Internet, that my web browser will systematically download on demand. The keyboard sequence control and U can be used in Microsoft’s Edge web browser to display the HTML code of the page that was returned from the website. For example, when I looked at the HTML code of the page loaded when I access [www.microsoft.com](http://www.microsoft.com), I saw numerous references to additional Internet content. One of those was the “template” that Microsoft’s website asks my web browser to download in order to properly display the web page: [//www.microsoft.com/onerfstatics/marketingsites-wcus-prod/west-european/mscomhp/\\_scrf/css/themes=default.device=uplevel\\_web\\_pc\\_webkit\\_chrome/fa-ea79ed/b3-d4e830/57-7b1339/37-e29aca/21-7d6c87/5a-e79275/54-41a2a0/65-478888?ver=2.0](https://www.microsoft.com/onerfstatics/marketingsites-wcus-prod/west-european/mscomhp/_scrf/css/themes=default.device=uplevel_web_pc_webkit_chrome/fa-ea79ed/b3-d4e830/57-7b1339/37-e29aca/21-7d6c87/5a-e79275/54-41a2a0/65-478888?ver=2.0).

It is common for these names to be complex and ugly, with embedded information that has

meaning to the web servers and web browsers on the various computers but are not intended for ordinary human interaction. Each of these components is downloaded, including graphical elements, cascading style sheets, scripts, templates, forms, etc. A web page being rendered by a web browser is a modern example of one embodiment of a “streaming on demand system” and is analogous in a number of ways to the system of the ’808 Patent.

52. A useful point to observe here is that a streaming on demand system need not use components that are located within a single storage system. Using my example of the [www.microsoft.com](http://www.microsoft.com) web page, it points to no fewer than six different Internet *domains* in the first 20 lines after the web page title. It is unlikely that mem.gfx.ms, c.s-microsoft.com, and assets.onestore.ms refer to the same data storage locations. Thus, these likely represent distinct servers, located in various sites around the world, with their own distinct storage.

53. The benefit of a streaming on demand system is that it provides the ability to obtain exactly the set of things that you need, when you need them, without requiring that you keep a copy on your local computer. The ability to download on demand only those pieces required is important because “the Internet” is sufficiently large that it is impractical to store an entire copy of everything you could download on your computer system, particularly before you can access any of it. Fortunately, that is not necessary, and we can instead download pieces of the Internet to your local computer to satisfy your specific content “on demand”. After I access [www.microsoft.com](http://www.microsoft.com), several of the static components of that website are stored on my local computer, in a cache. Caching is important in understanding both the ’808 Patent and the prior art literature cited by Microsoft, as caching is a common mechanism in computer systems, realized in many distinct ways. Note that none of the cited patents invented caching, as it is a fundamental idea used in computer systems decades before the time of even the oldest of the patents considered.

54. The term “**data packs**” should be construed to mean portions of a data file that correspond to objects with specific information content meaningful to the streaming application. My definition differs from the definition offered by Microsoft. The term “data pack” is not a term of art from the perspective of a POSITA. I draw my definition of the term “data pack” from the specification, which defines a “data pack” to be equivalent to an “asset” (’808 Patent Column 7, Lines 1-21, and Column 8, Lines 5, 61, 64, & 67, and Column 9, Line 27, and Column 10, Lines 26-31 & 60, and Column 11, Lines 31, 38, & 45). The term asset as used in the ’808 has a particular meaning derived from the field of computer games, though I note that even Microsoft’s own home page today uses the term “asset” with the same meaning as it is used in the ’808 Patent. The term asset has specific meaning to the streaming application because an asset has structure known to the application and is used to create a composition that controls the presentation of information to the human user. In a traditional file system, because the data contents of a file are opaque (¶31), the way files are subdivided by using a fixed sized unit, typically related to the ordinary size of physical media, to manage the file in smaller units.

55. For many decades, the standard size of data storage on hard disks was 512 bytes (the “sector”); while the value 512 was not a hard requirement, it was the most commonly used value. As disk drives became larger the sector size on hard disks has grown to 4096 bytes. The pervasiveness of 512 bytes means that disk drives with the larger 4096 byte units had to provide legacy compatibility to allow reading and writing in 512 byte units (Exhibit 2016). The “fixed size block” paradigm is an inherent part of ordinary thinking in the storage community. The ’808 Patent’s definition explicitly distinguishes itself from the ordinary fixed size block approach because of the additional insight from the game community.

56. The distinction between using the model of data as opaque fixed sized blocks versus variable sized blocks of data with specific context is not an artificial one: it is quite important to the ability for the system of the '808 Patent to function efficiently. Games are constructed by combining thousands of “assets” into a small number of distinct files within the storage system (“asset container” in the language of the '808 Patent. These assets are needed by the application to properly present content to the user. The specific location of a given asset within the distinct files (“asset containers”) need not be “close to” other assets the program requires at the same time.

57. The way applications are assembled is like the way that web pages are constructed: there is a list of distinct elements that must be combined together to form the entirety of the application – it is the list of distinct elements that corresponds to the “assets” of the '808 Patent. The simplest model of running an application is for the application to be entirely installed on your local computer, and then executed from that location. For small applications, downloading and installing an application prior to use is a simple and acceptable method for gaining access to the application. For large applications, the time to download and install the application increases. The important observation here, then, is that some applications can be quite large. Microsoft Visual Studio, for example, can be more the 20GB in size. Another example is Activision/Blizzard’s popular “World of Warcraft” game, which is more than 60GB in size. Both use a simple streaming download model, but Microsoft Visual Studio does not allow use of its content until the selected content has been downloaded and installed. World of Warcraft permits the user to begin playing the game with only a small amount of the content downloaded while it continues to download. The difference in user experiences here is palpable: it can take hours to download and install Microsoft Visual Studio while the World of Warcraft player can be playing within just a few minutes. In a business environment, that time is unlikely to be an impediment to using the business

application. For a game, excessive download and installation time can easily cause an interested potential player to abandon the process, as most games are free to download and try and recreational game players have a vast array of games from which to choose. The different behavior of a person downloading and installing a game to their local computer at home versus that same person's behavior downloading and installing a business application to their local computer at work is fundamental. When I install Microsoft 365 Apps for Business to my local computer, my company has dictated the tools that I am using, and my company is paying for my time to wait and install them. When I am home downloading a game, if it takes longer to reach the point that I can play than the time required to use the bathroom and get a snack and beverage, I will feel the game has failed to meet my needs for entertainment. A game on my home computer competes against thousands of other alternatives and my employer is not dictating that I must play a specific game. Thus, the game that is "ready to play" quickly by efficiently implementing streaming download has an inherent advantage over a game that requires the traditional download and install. Further evidence of the desire to minimize the time to download content from the Internet, including games, is apparent in the rapid adoption of residential broadband networking, which started long before the global pandemic forcing work-from-home.

58. Application program assembly is a complex task of picking the pieces of the application from a variety of locations: binary "object" files, which represent output from the tools that programmers use to write the programs, libraries of object files that have been previously generated and then combined, frameworks that are used to simplify the tasks associated with programming, as well as program data that is used as part of the program execution. When assembling the program content, it is uncommon for the tool that performs assembly to associate what is needed at execution time with its placement in the output files. The way in which programs

are assembled is important in understanding the critical nature of the '808 Patent's teachings regarding "data packs" being comparable to "assets". The definition of the boundaries of assets is typically discarded as part of packing these assets together in to a small set of files (Exhibit 2008). The location of these assets within the files has no relationship to the temporal locality of when these assets are needed by the application.

59. If a streaming application requires 200 unique assets in order to render a specific graphical page for the user, it is likely that it will need to access 200 different locations, across the set of files that correspond to the application. If the average size of these assets is 60 bytes, it would require reading 12,000 bytes of data. If these assets needs to be downloaded from the Internet (as part of the "streaming on demand" application), doing so would generate 200 separate requests, one for each asset required. If the streaming on demand system were to "round up" to a 4KB boundary, each read would, on average, require 4036 **extra** bytes that are not useful in satisfying the current request. Thus, instead of requesting 12,000 bytes of data, the streaming on demand system using 4KB blocks would need 819200 bytes, or 807,200 extra bytes. Such a read amplification constitutes a 67x increase in the amount of data downloaded. Equally important, increasing the amount of extraneous data downloaded while performing on-demand streaming increases the amount of time required to receive it, process it, and can lead to delays in rendering. The slower the network, and higher the latency of these data transfer operations, the lower the quality of the gaming experience. While high quality broadband network service is particularly important for interactive programs, it impacts all streaming on demand applications.

60. Thus, the '808 Patent teaches that instead of downloading blocks, a streaming on demand application should download assets. Assets are not fixed size; assets are exactly the correct size of the data that the application program needs. Further, assets can be present multiple

times within a streaming on demand application. Updates to such a streaming on demand application are likely to change the location of an asset, even when the asset itself has not changed. By teaching us to use a lookup table to map assets to local and remote locations, the '808 provides a novel solution to the problem of streaming on demand application delivery.

61. The term “**minifilter**” should mean a standard Windows kernel mode device driver that registers with the Windows filter manager. A standard kernel mode device driver is one that provides a declared entry point, typically “DriverEntry” because that is the default entry point name when using the standard Microsoft tools for Windows kernel mode device driver development and provides at least some nominal service to the Windows operating system. For example, the Windows filter manager is itself a standard Windows kernel mode device driver. It contains a DriverEntry routine, which is invoked by the Windows kernel as part of dynamically loading it into the computer memory and configuring it to interact with the Windows operating system kernel. In addition, the Windows filter manager also creates distinguished names that can be utilized by application programs for communicating with it, as well as creating “device object” that it then “attaches” to instances of existing file systems. The creation of names and objects and attaching the objects to file system objects is the ordinary function of a “legacy file system filter driver” in Windows and is a necessary part of filter manager’s functionality. A minifilter may call any kernel mode service that is generally available to kernel mode device drivers and is not restricted to using only the interfaces or services provided by filter manager. Thus, while the filter manager will invoke it for operations that the minifilter has requested, the minifilter is not restricted to only those operations. However, for the purposes of my analysis, I have adopted Microsoft’s definition of the term “minifilter”, because even using my more correct definition, my analysis remains the same.

62. The term “preoperation callback” means a registered call that is invoked prior to the I/O request completion. Note that the status of the operation is indeterminate when the preoperation callback returns to the caller, which means that it may be completed prior to that return, or processing might still be continuing when the preoperation callback returns to the caller. That an I/O operation can be completed to the application prior to return from the preoperation callback is a fundamental feature of the standard Windows driver model, in which I/O operations are managed asynchronously. One potential ramification of the asynchronous I/O model in Windows is that a “postoperation callback” can be invoked before the “preoperation callback” has completed. While my definition is different than Microsoft’s less correct definition, for the purposes of my analysis I have used Microsoft’s definition of “preoperation callback” because my analysis remains the same, even with their less correct definition.

63. The term “load before” when used in the context of standard Windows kernel mode drivers matches the ordinary English meaning of the term “load before”, in which it describes the order in which drivers are loaded by the operating system. However, one of the goals of the Windows filter manager was to permit the order of invocation to be independent of the order of loading. While technically incorrect, the term “load before” might be inappropriately used to mean “invoked before”. Thus, a POSITA would only know the documented meaning, which is consistent with the ordinary English meaning of “load before”. However, I have used Microsoft’s definition of “load before” because, while technically incorrect, it is more favorable to the interpretation of the ’808 Patent than my definition.

64. The phrase “a size indicator” means an explicit size or length associated with a data pack. I distinguish an explicit size or length from an implicit size or length that is based on a fixed

value, because such an indicator could consist simply of an offset relative to the fixed value, which is not consistent with the specification of the '808 Patent.

65. The phrases “address” and “addresses” mean a mechanism for specifying a resource available to a computer system, locally or remotely that a subset of the expressive ability of the uniform resource identifier (URI), as defined in IETF Request for Comment (RFC) 3986, published in 2005. Note that the definition of the URI is based upon the uniform resource location (URL), which updated IETF RFC 1738, published in 1994. Thus, the definitions in these documents would have been known to inventors of the '808 Patent as well as the authors of all of the prior art citations offered by Microsoft. (Exhibits 2119, 2120). For example, the name <https://www.microsoft.com>) is a valid URI. My definition of the terms “address” and “addresses” is consistent with the specification of the '808 Patent: “The addresses in the table may include addresses for a local source such as a hard disk, an optical storage medium (e.g., CD or DVD), a nonvolatile memory (e.g., a flash RAM), or a magnetic storage medium (e.g., a diskette), and an address for at least one remote source for a second data pack such as a server (e.g., ftp and/or web server) and a peer computing device. The minifilter requests to retrieve a required data pack from a first source corresponding to a first address in the table depicted in the figures or the order of steps or types of components, configurations, ornamental aspects or proportions shown in the figures.”

66. The specification of the '808 Patent clearly includes addresses of web servers, which suggests that the URI is a reasonably expressive definition of an address. The location of local resources on a computer can similarly be described by one, or more URIs.

**D. Person Having Ordinary Skill in the Art**

67. For purposes of analyzing the '808 Patent and the other prior art references provided by Microsoft, I have used Microsoft's definition of a person having ordinary skill in the art ("POSITA"), which I have used equivalently to Microsoft's use of the term "Skilled Artisan."

**VI. THE MICROSOFT OFFERED PRIOR ART FAILS TO DEMONSTRATE THE '808 PATENT IS OBVIOUS**

**A. Summary of Microsoft's Prior Art References**

68. Microsoft offered five prior art references: Publication No US 2006/10117018 to Christiansen ("Christiansen"), U.S. Patent No. 6,574,618 to Eylon ("Eylon"), U.S. Patent No. 6,993,603 to Pudipeddi ("Pudipeddi"), U.S. Patent No. 7,617,253 to Prahlad ("Prahlad"), and U.S. Patent No. 7,496,565 to Thind ("Thind"). I have reviewed these prior art citations, and my analysis explaining why, in my opinion, the cited prior art fails to render any of the claims of the '808 Patent obvious.

69. Christiansen teaches a file system minifilter based solution to construct a caching solution, in which files organized using a standard hierarchical file system structure are selectively downloaded and cached to a computer by mirroring the structure to an existing local file system. Thus, Christiansen teaches that the original hierarchical file system structure is mirrored, ensuring that an application may access that content as if it is entirely present, even though it is not. When accessed, the file content is fetched. As I have previously noted, the concept of dynamic on-demand fetching of data is one with which I am personally familiar because I was involved in building distributed file systems, which provide similar functionality.

70. Eylon teaches an application caching virtual file system solution, implemented on a client computer, which caches application blocks ("streamlets"). Eylon's system uses a set of files, which are organized using a standard hierarchical namespace model on a server computer,

the structure of which is mirrored on the client computer. Eylon's system may store those fixed size blocks ("streamlets") on the client computer by downloading them as fixed sized units ("streamlets"). Eylon's virtual file system keeps track of which streamlets for which files are present on the client computer and thus also which streamlets must be fetched from the corresponding file on the sever computer, if needed by the application running on the client computer. Eylon also teaches that the server computer can pro-actively push content to the client computer prior to the client computer reaching a point where it is needed.

71. Pudipeddi teaches the implementation and potential use of a file system filter manager, such as the Windows "file system filter manager", which is embodied by the dynamically loadable Windows NT kernel mode driver commonly referred to as "filter manager" or "fltmg.sys".

72. Prahlad teaches a mechanism for constructing consistent replicated copies of a set of data files by using a log with entries indicating points where the data is consistent and can thus be used as part of a data replication scheme.

73. Thind teaches a mechanism for maintaining "namespace consistency" by utilizing a file system filter driver to monitor operations that might impact the file system namespace and updating the file system filter's internal state to reflect those changes.

**B. Analysis of the '808 Patent in Light of Microsoft's Prior Art References**

74. In my analysis, I have used the standard of obviousness that I set forth in ¶¶23-26.

75. Claim 1 of the '808 Patent reads: "A streaming on demand system comprising an on-demand requester object installed on a computing device, said on-demand requester object being configured to receive I/O requests on behalf of an application for which data is available in data packs for streaming delivery, said on-demand requester object being responsive to application

I/O requests, said application I/O requests corresponding to sequential and non-sequential data packs, said data packs being portions of a complete data stream, said I/O requests determining an order in which data packs are sought for the application, said on demand requester object comprising a minifilter associated with a filter manager in an I/O stack, said filter manager being configured to call the minifilter, said minifilter having a pre operation callback routine for the I/O request registered with the filter manager, and said minifilter being configured to receive each I/O request from the application, said minifilter being further configured to reference a table that includes names and paths with at least one offset address and length where each data pack required to fulfill each I/O request is located, said minifilter being further configured to determine if the data pack has been streamed to the system, and said application being a program configured to use the data packs that fulfill each I/O request when the data packs are available on the computing device.”

76. In considering the ’808 Patent over Eylon, I note that there are similarities in the problems that both attempt to address, although the ’808 Patent and Eylon do so in worlds with very different technological considerations, different solution models, and different specific methods of addressing those problems.

77. Eylon’s primary focus is on the streaming transmission of a set of files that constitute an application. I previously described the concept of a “file” (¶29), which, in my opinion, is consistent with the use of the term “file” in Eylon. Microsoft has previously also considered the meaning of the term “file” (Exhibit 2029). An application “file” typically consists of both executable code, which are the instructions that the computer processor executes, and application data, which are used as part of the program execution. There is a certain ambiguity in the term “file” because a POSITA views files as being abstract and other similar abstract terms,

such as “object”, or “container”, may also be used interchangeably because these terms similarly are abstracted from consideration of the actual data content of the term being used. To a POSITA, these terms would suggest the variable sized nature of a specific instance of the abstract term. In other words, to a POSITA a file, object, or container has some arbitrary amount of opaque data, which is likely to differ between any two files, objects, or containers. Similarly, when considering memory, a POSITA has likely learned the term “page”, which is used to represent a fixed sized unit of memory allocation that is supported by the underlying operating system and hardware. For example, on a Windows PC using an Intel or AMD base central processing unit, that is a fixed 4,096 bytes (4KB). The motivation to used fixed size units is that it greatly simplifies the process of tracking them: The Intel or AMD CPU must traverse an in-memory hierarchical table to resolve the addresses used by applications and the operating system into references to the underlying physical memory (e.g., Dynamic Random-Access Memory, also known as DRAM). These “page tables” themselves are constructed of such fixed sized pages. Similarly, storage devices, including hard disk drives, solid state drives, and optical storage devices, divide their own storage space up into a series of fixed size units. One commonly used “sector size” for modern hard disks is 4,096 bytes (4KB). Application programs that run on the native processor are typically constructed by specialized tools, including compilers, linkers, and library managers. These tools take the programming instructions (“code”) that was written by one or more software developers and combine it with potentially hundreds of other pieces of code that have been written and compiled previously. A linker is the tool that assembles the program components together into an “executable image”. Executable images must be loaded into memory in order to allow the computer processor to “run the program”. The linker tool is aware of the “page size” supported by the underlying computer and may reorganize the code within the application program to

conform to those page sizes. For example, the Intel and AMD processors utilize variable sized instructions. The Microsoft compilers will not generate instructions that straddle page boundaries because there have been bugs in certain CPUs that did not handle the straddling of a single multi-byte instruction across page boundaries properly. The tools that Microsoft uses when assembling Windows are given explicit execution information so that code routines that frequently run can be reorganized to live on the same page. Organizing code so that frequently executed code is grouped onto a small set of pages creates important processor efficiencies because processors rely upon caches to achieve optimal performance, much like storage systems use caching to achieve optimal performance. Indeed, Eylon's system exploits the grouping of code onto a small set of common pages: if the underlying application building tools have organized code that is frequently used onto a small set of pages, there are inherent benefits to a streaming application delivery system such as Eylon's to sending individual blocks, since many of them may contain code that will not be needed immediately. Further, application programs will have a set of data, whether in blocks or data packs, that applications need in order to initialize and reach a point where an application can interact with the human user. The data, whether in blocks or data packs, needed to start up is likely to be a small subset of the total content. Both the '808 Patent and Eylon have realized that only a small portion of the data is needed to initialize and once that data has been downloaded the application can begin running. The ability to improve performance by loading application data in the order it was needed is one that was known prior to Eylon (Exhibit 2035). A POSITA would understand that it is fastest to download the critical data needed to initialize the application first. Eylon's observation that the server computer could push data to the client computer, if the server knew what data the client might need is part of a long-held observation that we can build fast, efficient systems with slow storage when we can ship the needed data so that it arrives "just in

time” for the computer system to use it; indeed, a common model for memory and storage is that they are equivalent (Exhibit 2039). Eylon’s system does not require such predictive intelligence because we know that such is not always required. We can gain such insight when we know more about the applications and their usage patterns.

78. These general terms: page, block, streamlet, sector, data pack, file, container, object, etc. must be judiciously used within their context. If we were to insist on taking the broadest interpretation, which appears to be the approach that Microsoft suggests, we would have no storage patents: Alan Turing defined storage in its most broad and abstract sense in 1936 (Exhibit 2041). Instead, we realize that specific implementations of storage have different benefits and costs and teaching a specific embodiment of the very abstract concept of storage. Alan Turing proved, in the mathematical sense, that a single infinite tape drive satisfies the storage requirements for any possible computing device, and further that multiple infinite tape drives are equivalent to individual tape drives. However, we know that in a specific embodiment of these concepts there are real benefits to using multiple devices rather than a single device. Thus, when considering these abstract terms, I have taken care to consider the specific embodiments described within each of these patents.

79. Microsoft argues that the term “streamlet” (Eylon) and “data pack” (’808) are equivalent terms of art. I would not expect a POSITA to know the specific meaning of these terms beyond understanding that, in an abstract sense, the terms are similar. A casual reading might encourage assuming these two terms are equivalent, the conclusion the term “streamlet” and “data pack” are the same does not bear up under scrutiny. A more careful reading makes it clear that Eylon is concerned with the preservation of an existing set of files, in which each file is deconstructed into a set of fixed-sized blocks, which Eylon refers to as “streamlets”. Eylon

Column 3, Lines 56-57 says: “[m]ore specifically, the application to be executed is stored as a set of blocks or ‘streamlets’ (parts into which the application has been divided) on a server. In a preferred embodiment, each streamlet blocks corresponds to a data block which would be processed by the native operating system running on the client system were the entire application locally present. For example, standard Windows systems utilize a 4 k code page when loading data blocks from disk or in response to paging requests. Preferably, each streamlet is stored in a pre-compressed format on the server and decompressed upon receipt by the client.” Thus, what Eylon teaches is to divide up our application files into *pages*. Dividing data up into pages, which map to how a storage system that treats data like opaque blocks, makes sense for Eylon’s problem domain.

80. The decision to used fixed size “streamlets” makes sense within the context of the specific problem that Eylon is trying to solve, namely supporting streaming applications. As such, an application is accessed in fixed sized units (“pages”) and that is Eylon’s preferred size for a “streamlet”. Hard disk drives in the relevant time frame most commonly used 512 byte sectors, as I described previously (¶31). Thus, 4KB was a good choice for the page size of Eylon because it represented a multiple of 512 byte sectors, which also mapped well to obtaining optimal performance from the storage devices of that era. Eylon’s system is beneficial because, as programmers add functionality to applications, the application becomes larger, yet human users do not access all those features at the same time. Thus, only loading in the application code, which is organized in fixed size pages, as it is needed (or anticipated to be needed), yields a useful and tangible benefit.

81. The ’808 Patent also teaches about application streaming, but it does so using a different type of application, namely games. The breadth of material in a game potentially supports a vast virtual world of graphical elements that form the basis of dynamically generated content.

However, unlike the business application programs that were of particular concern to Eylon, where it makes sense to divide code up into page sized chunks, the games of the '808 Patent consist mostly of data elements that have no relationship to the memory layout of the underlying computer and instead are related to the needs of being able to render graphical information to the human user. The code portions of a game are typically quite small in comparison to the graphical data portions of the game. That most game data is graphical elements ("assets") is distinctly different than the environment of Eylon. For example, if I use the Adobe tools for manipulating graphical elements such as Photoshop and Illustrator, those are primarily *applications* and would thus benefit from the simple block oriented model taught by Eylon.

82. Another important observation that the '808 Patent teaches is that applications which utilize assets are actually not interested in the *location* of those assets but rather the *identity* of those assets. For example, I previously (§§50-52) used the standard Internet web browser to describe the concept of assets and how these assets are used to render a visible web page. A standard Internet web browser typically stores those assets in a cache directory of the local system. The user is unaware of where or how these are stored. The standard Internet web browser could store multiple assets into a single file, or into multiple files, or even in a distinct object store. These mechanisms were all known prior to Eylon (Exhibit 2023) as were the disadvantages of using traditional file systems for storing web objects (Exhibit 2022). An important benefit of focusing on assets rather than location is self-describing assets can have multiple versions, such as a graphic image in different sizes and quality levels. File systems are unaware of the semantic context of the data contained within the file and thus do not preserve that structure or context.

83. As I pointed out earlier (§41) the network availability in the time of Eylon is very different than the time of the '808 Patent (Fox 1996). No one in 1998 would worry about dynamic

streaming of large game images because it was impractical over dial-up connections. Thus, the actual problem Eylon was addressing does not even consider the actual problem that motivates the '808 Patent.

84. The “data packs” of the '808 Patent do not relate to fundamentals of the underlying storage. Instead, “data packs” relate to the meaning of the data that is stored within the storage itself. Thus, the '808 Patent teaches something that Eylon does not: namely, the use of application specific knowledge when building our streaming delivery system. The specification of the '808 Patent makes the benefits of using application specific knowledge of how program assets are access clear in several ways. First, the specification of the '808 Patent teaches that we must look up *each* data pack, which is necessary because data pack sizes are unrelated to the storage system sizes. The limitation that we must look up each data pack exists in the claim because we need the offset and length to map from the opaque storage location to the logical identity of the “asset” or “data pack”. By mapping the offset and length to an identity, the '808 Patent teaches a way to map from the opaque data of the files within the file system to the application asset. In other words, mapping the offset and length to an identity of the data block permits the streaming on-demand system to understand the context and semantic meaning of that asset once again, which permits the construction of a more efficient streaming on-demand structure.

85. The use of “data packs” requires the claim limitation of maintaining the mapping table because maintaining a large map of variable sized objects requires more complex data structures, more memory, and more processing time. Using fixed size blocks permits simpler data structures, which require less memory and less processing time.

86. The specification of the '808 Patent explicitly adds a two-stage lookup of an asset, because the inventor of the '808 Patent, has realized, due to his computer game background, that

assets can be duplicated, as well as change location, even though assets do not change content – when an asset changes content it is no longer the same asset. The inventor observed that there is no benefit to re-downloading the same asset, simply because it has changed its logical location, within the original file, or even to a different file.

87. The movement of the asset is described in Column 7 of the '808 Patent. Eylon has no comparable teaching, which is consistent with Eylon's traditional storage model of treating application data as opaque blocks of data. Another example of the difference between the data context aware assets of the '808 Patent and the data opaque block structure of Eylon is that the specification of the '808 Patent distinguishes between the 4KB I/O operations of the Windows operating system, known as "paging I/O" operations, and the arbitrary offset and length I/O operations that an application may use (Column 7, Lines 50-55). Paging I/O fits Eylon's use of the term "streamlet": fixed sized blocks without context. The data packs of the '808 Patent do not fit with paging I/O because data packs use the specific details of the I/O operation from the application, as specified by an arbitrary offset and length. The specification of the '808 Patent explains how large block transfers are mapped into the correct corresponding assets ("data packs"): "If the MLAT reflects that the region (e.g., DATA.DAT starting at 10,000 for 200,000 bytes) has not been serviced, a source (i.e., an asset container) that has the required data is cross referenced. If the container is local, the data will be copied into the destination file (DATA.DAT). If the container is not local, a list of assets needed to service the request will be sent to the user mode application, which will then proceed through a list of media locations 324-334 checking for the assets" (Column 8, Lines 59-66). Note that the specification of the '808 Patent clearly identifies that blocks are mapped into assets, and that each asset may come from a distinct, separate location. The '808 Patent clearly distinguishes between blocks and assets: the "streamlets" of Eylon are not

equivalent to the “data packs” of O’Brien. Claim 1 of the ’808 Patent clearly identifies that “data packs” are “portions of a complete data stream”. Because a “data stream” is made up of distinct “data packs” and each “data pack” can map to a distinct location, whether local or remote, Claim 1 does not limit an embodiment to a correspondence between the local and remote file. Eylon and Christiansen both teach a correspondence between the local and remote file. Had Claim 1 intended to limit the mapping from a remote file to a local file, it should have used the term “file” rather than “data stream”, which is a collection of data packs, each of which distinctly references one or more locations, whether local or remote, from which the data pack contents can be retrieved.

88. Eylon teaches the use of a “virtual file system” for storing the data of their system. In the time of Eylon, the concept of a file system filter driver was certainly one that was well-known within the file system field, as I taught public seminars that discussed developing file system filter drivers, I routinely published publicly available information about developing file system filter drivers, and the commercial book publisher O’Reilly sold a book entitle “Windows NT File System Internals: A Developer’s Guide” since September 1997. Thus, the concept of a file system filter driver, distinct from a file system, was known to Eylon. However, Eylon chose to exclude the distinct use of an existing file system, combined with a file system filter driver from their specification.

89. I reiterate that a file system filter driver and a file system, regardless of type, are distinct entities. A file system filter driver has, as its requirement, at least one file system. The requirement that a file system filter reside on top of another file system filter or a file system filter driver is inherent in the name: one cannot filter something unless that thing exists. Unlike a file system filter driver, a file system does not require the existence of a file system filter driver. Microsoft themselves have shipped versions of Windows that contain only file systems, without

file system filter drivers. Thus, these are different terms, and it is not reasonable to claim that a file system and file system filter driver are equivalent.

90. Because Eylon is different than the '808 Patent, there would be no reason for a POSITA to combine it with any of the other prior art references that Microsoft has provided.

91. Further, both Christiansen and Eylon implement caching systems. Indeed, were one to use a Windows embodiment of the system that Christiansen teaches, it would be possible to run application programs from such a system, without the use of Eylon. Thus, the advantages of combining Christiansen with Eylon would not be obvious to a POSITA, since there would be no obvious benefit for doing so.

92. Finally, while Christiansen and Eylon are teaching different ways to implement content caching systems using on demand streaming download, Christiansen and Eylon do so using different techniques that logically interfere with one another: Christiansen is teaching the use of a file system filter, Eylon is teaching the use of a file system. A POSITA is unlikely to combine the teachings of these two patents because the two patents achieve similar purposes but with different, inconsistent techniques.

93. Thus, based upon my analysis, it is my opinion that Claim 1 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

94. Claim 2 of the '808 Patent reads as follows: "A streaming on demand system according to claim 1, said minifilter being configured to load before compression and filter minifilter drivers in the I/O stack."

95. Claim 2 is a dependent claim that depends on Claim 1. I understand that Claim 2 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach the limitations of Claim 2.

96. Microsoft appears to read the text of Claim 2 to include the term “encryption”. I have adopted Microsoft’s reading of Claim 2 in my analysis, and consider that the corrected text, as per Microsoft’s explanation, be used: “A streaming on demand system according to claim 1, said minifilter being configured to load before compression and encryption minifilter drivers in the I/O stack.”

97. Microsoft has asserted that it is obvious to implement the functionality of Eylon at an altitude higher than compression and encryption minifilters. I have implemented both compression and encryption filters and minifilters on the Windows platform and based upon my experience, I disagree with the “obviousness” of Microsoft’s assertion.

98. As previously stated, Eylon teaches the use of a “virtual file system” and not of a “file system filter driver”. Thus, in the system of Eylon, which is a file system, it is not possible for a filter or minifilter to be layered “below” Eylon, it would be required to be layered “above” Eylon.

99. However, using the Microsoft analysis, it is important to note that the system of Eylon is compatible with a filter or minifilter that implements encryption or compression residing on top of the virtual file system of Eylon, which highlights a key difference between the teachings of Eylon and the ’808 Patent, which further demonstrates that the ’808 Patent cannot be made obvious by combining Eylon with Christiansen.

100. Any file system or file system filter driver that supports data transforming drivers must be capable to efficiently support random access. The ability to support random access efficiently is one of the fundamental aspects inherent in the Eylon specification: a “streamlet” must be sufficiently decoupled from any data preceding that point that it can be used by itself.

101. The normal mechanism for achieving efficient random access to an encrypted or compressed file is to define a fixed size unit for encryption or compression. For example, the Windows NTFS file system, which incorporates both encryption and compression support within it, does so by defining a fixed size for such encryption and compression. For example, NTFS performs compression on 64KB units. However, the choice of 64KB is an arbitrary choice of NTFS and exhibits why the virtual file system of Eylon would not be subject to such considerations: the minifilter's altered data blocks would be mapped to the data agnostic "streamlets" of Eylon. While a filter using fixed sized blocks that are different than the size of the encrypted and/or compressed data might lead to a system in which some different number of streamlets would be stored or retrieved, Eylon would continue to function properly because it has no dependence on the *contents* of the streamlets.

102. A system that combined an encryption or compression file system filter with Eylon's virtual file system would lead to precisely the opposite of what Microsoft proposes: an encryption and/or compression filter that logically was above Eylon, yet still provided both streaming on demand delivery of application programs (the purpose of the teachings of Eylon) combined with encrypted or compressed data.

103. Christiansen teaches the implementation of a caching web content system, and it would benefit most if the data that its filter received is compressed. Microsoft spent considerable effort to enable the ability to store and retrieve compressed data in the NTFS file system, for example, by permitting a special mode for the Server Message Block (SMB) redirector to read compressed content from a remote file server and write it directly into the NTFS file system. In that way, when an ordinary application accessed the file, NTFS would retrieve the compressed data, whether in memory or on the storage controlled by NTFS, decompress it into the file system

data cache, and then serve those file contents to the requesting application. No filter is required to support compression and decompression with NTFS, but if a filter is present, it would have access to the internal Windows kernel mechanisms used to read and write compressed data into NTFS.

104. The use of compressed data, rather than uncompressed data, is particularly important over wide area networks (WANs) because compression can substantially decrease the amount of network bandwidth required to send the data. The precise amount of benefit that transmitting compressed data provides is directly related to the information density of the corresponding data (Exhibit 2018).

105. I note that the benefits of data compression are conceptually like the teachings of the '808 Patent, each of which would decrease the amount of network traffic required to satisfy an application data request. However, the teachings of the '808 Patent are far more efficient because it only fetches the specific "data pack" ("asset") required by the application, not the data surrounding the "data pack" ("asset"), which is required by encryption and compression filters, encrypting and compressing file systems, as well as by the teachings of Eylon and Christiansen.

106. Finally, I note that Microsoft argues that combining the teaching of Pudipeddi with the teachings of Eylon and Christiansen is obvious because a minifilter residing above the encryption/compression filters would be simpler. I disagree with Microsoft's assertion because it is not consistent with my own experience in implementing these types of filters.

107. In my experience, a minifilter that resides below an encryption or compression filter is normally simpler than one that resides above such filters, due to the way that these elements must be implemented on Windows. It is easier to implement a filter below an encryption or compression filter because encryption and compression filters in Windows must convert all read and write I/O operations into a single type of I/O operation, namely non-cached, fixed-sized I/Os.

Typically, the fixed size chosen is the page size, or a multiple of the page size. Thus, a filter situated above an encryption or compression filter must be cognizant of paging I/O and application I/O operations, both cached and non-cached, so the I/O operations can be properly intercepted and handled, including the I/O operations that the encryption/compression filters themselves generate. A filter situated below an encryption or compression filter would need to only be concerned with non-cached I/O operations that will ordinarily be fixed block size. Filters below these would be focused on managing blocks and not data packs, because the latter require awareness of the internal structure of the data itself, while the former is not affected by it.

108. Again, the use of fixed sized blocks for data management is consistent with the teachings of Eylon and Christiansen, which do not teach distinct handling of application I/O requests distinct from paging I/O requests. The block structure of Eylon and Christiansen is consistent with their treatment of files as being opaque entities without form or structure. The '808 Patent materially deviates from the opaque block-structured model taught by Eylon and Christiansen by exploiting knowledge of the structure of the file data to define the boundaries of the "data packs". These "data packs" can then be streamed to the local system, without including additional data that is not needed to satisfy the immediate request. Similarly, the '808 Patent can map a data pack request to the optimal location for retrieving those data packs, such as might occur when the same asset is present multiple times in the files of the streaming application.

109. If I were to implement the virtual file system of Eylon, an encryption or compression filter must be located on top of the virtual file system of Eylon because the file system implementation is the only required element of the stack of filters on top of a file system. The file system of Eylon is always located at the bottom of the stack of file system filters. The file system of Eylon is the same as for other Windows file systems, such as NTFS, that are present in systems

such as Microsoft Windows. Because Eylon is a virtual file system, it would exist below encryption and compression filters, which demonstrates the error in Microsoft's analysis. That error is not surprising considering how Microsoft confused a file system with a file system filter.

110. Thus, if I were to implement a block-oriented minifilter of the type taught by Christiansen, I might choose to implement it below a compression filter, to ensure that compressed data is transmitted over the network.

111. Similarly, if I were to implement a block-oriented minifilter of the type taught by Christiansen, I might choose to implement it below an encryption filter, to ensure that the data remains encrypted at rest on the storage system, as well as on the local computer that is caching the data contents because it would prevent information exposure. "End-to-end" encryption is important for environments where sensitive data needs to be transmitted over the network. Retrieving encrypted data from the file system, with or without the help of a file system filter or minifilter is not the only way to protect such information, but I have developed systems that in which a filter can choose to provide encrypted or decrypted data depending upon the specific request. My system permits storing encrypted files on an untrusted storage system as well as transmitting encrypted files on an untrusted network from a trusted storage system by filtering above Microsoft's own SMB file system client and below Microsoft's own SMB file system server.

112. The ability to work above or below an encryption or compression file system filter or minifilter is distinguished from the teachings and claims of the '808 Patent. An embodiment of the '808 Patent, using a file system minifilter, with its variable sized data pack ("asset") model cannot operate below compression and encryption filters because, by their very nature, compression and encryption filters modify the layout of data within the file in such a way that the only way to operate is the block/page/sector/streamlet model taught by Eylon. Because the '808

Patent defines blocks in terms of the data streams needed to make it up, a block-oriented encryption or compression scheme would require fetching all of the data packs for a given encryption or compression block, which would eliminate the improvements of the '808 Patent.

113. An example of the block-structured orientation in encryption is given by looking to the most used symmetric data encryption mechanism: the Advanced Encryption Standard (AES) as published by the US National Institute of Science and Technology (NIST) in FIPS 197, which was first published in 2001. In the time of Eylon, the Data Encryption Standard (DES) would have been well known, as FIPS 46, which defined it, was published in 1977. FIPS 46-3 was retired in 2005. In my experience, a system constructed to use DES can be modified to work with AES.

114. Storage solutions use a mechanism known as “cipher block chaining” (CBC) with a fixed block size. “Cipher Block Chaining (CBC) XORs each message block with the ciphertext of the previous block before encrypting it; an initialization vector, possibly public, is XORed with the first message block before encryption. This does require padding the message to a multiple of the block size...” (Exhibit 2030). Storage solutions must permit “random access” within the storage device and files so periodically the encryption logic must restart the CBC. Typically, the block size chosen by the implementor of a CBC based encryption scheme uses a value that is compatible with the encryption algorithm, which has some minimum output size, and the data unit of the storage: a block, a cluster, a sector, or a streamlet. A system supporting variable sized storage units, such as files, must perform special processing for the last block of the file as the encryption unit may not align with the amount of data in that last block; the need to “pad out” the last block of a file might seem to be a simple detail but it is a significant source of complexity in building encryption and compression file system filters; some of the patents on which I was a co-inventor deal with efficiently solving the challenge of implementing block structured encryption and

compression in a file system filter driver. One of the responsibilities of an encryption filter is to handle these issues to ensure correct application program behavior, as well as file system minifilter drivers located above the encryption filter.

115. Because the systems of Eylon and Christiansen are not dependent upon the internal structure of data within the file, a system embodying Eylon or Christiansen can be loaded below the encryption and compression filters of the system. The system described and claimed in the '808 Patent cannot do so, because it is dependent upon being able to access regions of the file that do not necessarily fall on storage related block boundaries.

116. I would further note that the level of knowledge and understanding of encryption and/or compression minifilters required to understand the issues I have described do not fall within the realm of abilities of a POSITA, using Microsoft's definition of "Skilled Artisan" because it is not part of the ordinary education to describe the details of how encryption and compression are implemented within the context of file system filter drivers. File systems, if taught at all, might be an hour of lecture in an undergraduate operating systems course. In my experience, file system filter drivers are not something that is taught in an undergraduate operating systems course. A graduate operating systems source, such as CS 6210 at the Georgia Institute of Technology or CPSC 508 at the University of British Columbia, both R1 level research institutions in computer systems, do teach about file systems, but do not teach about file system filter drivers, or storage layer encryption and compression considerations. Microsoft's definition also includes four years of relevant work experience as an option. However, even after four years of relevant work experience writing applications that can be streamed, on the Windows platform, an ordinary developer would not have an understanding of the challenges and trade-offs required to understand the limitations of building encryption and compression in file systems or file system minifilters.

117. Finally, I would note that combining Pudipeddi with Eylon would contradict Microsoft's own logic, because if Eylon is equivalent to the file system minifilter of Pudipeddi, it would not be able to exist below an encryption or a compression file system minifilter. Thus, using Microsoft's own analysis, Eylon is not a file system minifilter, because it must always exist below the encryption and compression filters.

118. There is no obvious reason to combine Pudipeddi with Eylon and/or Christiansen because Eylon is a file system, not a file system minifilter, and thus is fundamentally independent of the filters attached to it. There is no obvious reason to combine Eylon with Christiansen because Eylon and Christiansen implement the same logical system. Any combination of Pudipeddi, Eylon, and Christiansen fails to teach the limitation of Claim 2 of the '808 Patent that it must be layered above an encryption and/or compression file system filter or file system minifilter.

119. Thus, based upon my analysis, it is my opinion that Claim 2 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

120. Claim 3 of the '808 Patent reads as follows: "A streaming on demand system according to claim 1 said minifilter attaching to the I/O stack by registering with a filter manager in the I/O stack for I/O requests for the application for which data is available in data packs for streaming delivery."

121. Claim 3 is a dependent claim that depends on Claim 1. I understand that Claim 3 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

122. Thus, based upon my analysis, it is my opinion that Claim 3 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

123. Claim 4 of the '808 Patent reads as follows: "A streaming on demand system according to claim 1 said table including a size indicator for each data pack."

124. Claim 4 is a dependent claim that depends on Claim 1. I understand that Claim 4 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

125. In addition, I note that neither Eylon nor Christiansen teach the concept of including a size indicator *for each data pack*. Both Eylon and Christiansen are focused on files and fixed-size blocks. These solutions only require storing the size of the file; the size of their unit of data management ("streamlet size" or "block size") is a fixed unit and thus does not need to be stored separately. The need to explicitly store the offset and length for each data pack is due to the novel nature in which the '808 Patent is using knowledge of the structure of the file data to fetch only the specific information needed by the application using the streaming on-demand system. Thus, the '808 Patent maps (file, offset, length) into a corresponding asset identification and from that obtains the "address" of where the data is stored (container, offset, length). Eylon is oblivious to the structure of the data and focuses simply on the block-structure of the file. These are not equivalent.

126. Thus, based upon my analysis, it is my opinion that Claim 4 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

127. Claim 5 of the '808 Patent reads as follows: "A streaming on demand system according to claim 1 said minifilter being configured to pass on a read call if a required data pack is available locally and if all data in the required data pack has been received."

128. Claim 5 is a dependent claim that depends on Claim 1. I understand that Claim 5 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

129. In addition, Christiansen teaches a simple one-to-one mapping of cached content within the system to the original source. In contrast the '808 Patent teaches that if the "asset" is the same, we can retrieve it from any other copy of that same asset that is known to the requester object. For example, an update to the streaming application in the system of Christiansen would require re-downloading the entire content of the updated file, while the system of the '808 Patent allows mapping from the new location of assets from their location in older versions of the same files. Satisfying the need for the data pack from a local source of the asset is better than forcing the download of a file, or even a portion of a file from a remote location because it reduces both the time required as well as the amount of network bandwidth consumed. Neither Eylon, nor Christiansen teach using the internal structure of a file to more efficiently download its contents.

130. Like Christiansen, Eylon teaches a simple one-to-one mapping of cached content within the system to the original source. That Eylon and Christiansen are using a simple one-to-one mapping of remote file to local file is inherent in the fact that it is mirroring an existing application tree from the server computer to the client computer.

131. Thus, based upon my analysis, it is my opinion that Claim 5 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

132. Claim 6 of the '808 Patent reads as follows: "A streaming on demand system according to claim 1, said minifilter being configured to withhold a read call if a required data pack is not available locally."

133. Claim 6 is a dependent claim that depends on Claim 1. I understand that Claim 6 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

134. In addition, building upon the analysis in ¶128, for both the teachings of Eylon and Christiansen, the read is directly related to the corresponding data of the remote server, since there is a one-to-one correspondence between the local file and the remote file being cached by the local system. The '808 Patent claims a correspondence that is different than these teachings. For example, the '808 Patent teaches and claims the ability to satisfy the read for the data pack by fetching it from a different location that does not necessarily correspond to the remote source, such as part of a different file. The ability to satisfy the specific data pack from using an arbitrary source, local or remote, further demonstrates that the limitations of Claim 6 of the '808 Patent differ from the teachings of Eylon and Christiansen.

135. Another distinction between Christiansen and Claim 6 of the '808 Patent is that the specification of Christiansen teaches that data is fetched from the remote location; the local file system is not involved. Thus, the read is not withheld to the local file system, it is satisfied by the remote file system. These are subtle, but important distinctions in the terminology being used: Eylon teaches that the application is notified the request cannot be satisfied at the current time (such as by returning EBUSY) and Christiansen teaches that the read request is converted to a network read request. In contrast, the '808 Patent teaches and claims blocking the read request (in Windows, a call to KeWaitForSingleObject, for example) until the necessary data pack has been stored locally. Once it is stored locally, the original read request is then redirected to the appropriate portion of the data pack, which is stored locally.

136. Thus, based upon my analysis, it is my opinion that Claim 6 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

137. Claim 7 of the '808 Patent reads as follows: “streaming on demand system according to claim 1, said minifilter being configured to withhold a read call if all data in the required data pack has not been received.”

138. Claim 7 is a dependent claim that depends on Claim 1. I understand that Claim 7 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

139. Building upon my analysis in ¶134, I note that the term “busy” is not equivalent to “blocking”. Both POSIX (UNIX/Linux/macOS) and Windows have mechanisms for notifying an application when an I/O operation cannot be immediately satisfied. For example, by 1997, the POSIX real-time extensions were well known in the field (see <http://www.unix.org/version2/whatsnew/realtime.html>) and provided asynchronous I/O functionality. The ability to communicate the status of an in-process I/O operation is an important part of the concept of asynchronous I/O, which can be used by applications to improve performance and achieve greater parallelism. Asynchronous I/O is an old idea, certainly predating Windows, for example the select system call in UNIX, introduced before the time of Eylon, which I was using in my own programming work in the 1980s. Asynchronous I/O relies upon the underlying system to notify it when something “cannot be done now”. There are multiple mechanisms for implementing asynchronous I/O. Similarly, the Windows kernel implements an inherently asynchronous I/O system, as I explain in my book Windows NT Device Drivers (Windows NT Device Drivers). Asynchronous I/O was present in Windows NT at its initial release in 1993. Windows has enhanced its support for asynchronous I/O, such as the introduction of I/O

completion ports, introduced after Eylon, but before Christiansen and before the '808 Patent). In those systems, it is expected that the storage system will indicate that the request cannot be completed now. For POSIX, the error EBUSY can be used to indicate that a request cannot be completed now. For Windows, the status STATUS\_RETRY can be used to indicate that a request cannot be completed now. For asynchronous I/O operations, a typical operating system will permit cancelation of the in-progress I/O request, such as when it is taking too long, or the application terminates prematurely. When the operating system blocks the request, no status is returned to the application until the request unblocks. Thus, these requests cannot be asynchronous. Claim 7 of the '808 Patent's limitation that the I/O operation is blocked is not reflected in the teachings of Eylon and Christiansen.

140. Thus, based upon my analysis, it is my opinion that Claim 7 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

141. Claim 8 of the '808 Patent reads as follows: "A streaming on demand system according to claim 1, said at least one address including and address for at least one local source for a first data pack and an address for at least one remote source for a second data pack."

142. I note that in my analysis I have considered that the first "and" should be "an", so that Claim 8 of the '808 Patent, as corrected, reads: "A streaming on demand system according to claim 1, said at least one address including *an* address for at least one local source for a first data pack and an address for at least one remote source for a second data pack." [emphasis indicating my correction]

143. Claim 8 is a dependent claim that depends on Claim 1. I understand that Claim 8 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

144. I agree with Microsoft's explanation that Christiansen teaches mapping the local cached copy of a given file to the remote server from which it was fetched (Page 54, Section 8). "This mapping information is used to determine whether to fetch requested data from a local cache ("C:\CACHE") or a remote server ("\\SERVER\SHARE")."

145. Christiansen teaches that a local cached file is mapped to the remote file. However, Christiansen's teaching of mapping local cached file to remote original file is different than the mechanism claimed by the '808 Patent because a system embodying the '808 Patent maps the assets/data packs of which it is aware in the local file to a corresponding asset/data pack that may be stored locally or remotely, in the same or different file(s), or even in something that is not a file system, such as a key-value store or database. For a POSITA, it would be clear that the compositional approach of the '808 Patent, based upon the meaningful assets of the file and their locations, is different than the linear mapping between the client copy and server copy of the file that is taught by Christiansen and Eylon. Indeed, the "data packs" of the '808 Patent permit mapping back to the original assets, where those assets were distinct elements with specific meaning, prior to them being assembled into the files that are generated as part of the streaming application assembly process. Thus, using Microsoft's own analysis the teachings of Christiansen are distinctly different than Claim 8 of the '808 Patent.

146. The impact of Claim 8 and the teachings behind it in the '808 Patent are easily missed by a POSITA with direct experience in the storage arena, precisely because someone working in the computer data storage field is used to ignoring the data contents of files; someone with computer data storage experience does not consider the fact that the streaming application understands not only the layout, position, and structure of the assets/data packs, but also the contextual meaning of those assets. This contextual meaning is not ordinarily maintained once the

assets are packed into files. This is consistent with the ordinary use of files, assumed in the teachings of Eylon and Christiansen, to be oblivious to the internal structure of the streaming data files. However, the treatment of files as a set of opaque blocks without form or structure is not consistent with the '808 Patent, which teaches and claims that files have structure, said structure is based on data packs ("assets") and that said data packs/assets may be stored locally in a location other than using a trivial one-to-one mapping from the client computer's file copy to the server computer's file copy.

147. Christiansen is teaching a simple caching scheme. Indeed, the caching scheme is one that is so well understood the patent office rejected the application and ultimately Christiansen (and Microsoft) decided not to pursue the application. However, the '808 Patent does not teach and claim a simple one-to-one caching scheme. It teaches that files are composed of distinct elements: the "data packs" which are assets to the application program. The "data packs" of the '808 Patent, with their mapping back to the contextual meaning of the original asset, their per-data pack location information, and the composition of blocks ("data streams") by finding the set of data packs that cover the block distinguishes the teachings and claims of the '808 Patent from the teachings of Christiansen. The '808 Patent teaches and claims that the internal structure of information within the files can be identified and can itself serve as the unit of management, including caching.

148. Microsoft's analysis regarding Claim 12 ignores the difference between the content agnostic linear mapping of files transferred in fixed sized blocks taught by Eylon and Christiansen, to the content aware non-linear mapping of files transferred in variable sized data packs claimed by the '808 Patent. Failing to recognize that "data packs" mapping back to the contextual meaning of the original asset, the per-data pack location information, and the block composition using the

set of data packs that cover the block is different than a one-to-one mapping of file and blocks between a cache client and a storage server helps explain why a POSITA would not have understood how the claims of the '808 Patent could be realized from the teachings of Eylon and Christiansen. The difference is a unique and non-intuitive (to a person familiar with only storage) use of variable sized data packs and mappings to local and/or remote instances of those data packs that is taught and claimed by the '808 Patent.

149. Microsoft's analysis regarding Claim 12 underscores its misunderstanding of the difference between the content agnostic linear mapping of files transferred in fixed sized blocks taught by Eylon and Christiansen, to the content aware non-linear mapping of files transferred in variable sized data packs claimed by the '808 Patent when Microsoft states: "[t]hird, Eylon states that '[w]hen the desired *page* is not available [locally on the client system], a fetch request for the needed data is sent...to the server 12.'" [emphasis added] Microsoft's inability to understand how "pages" differ from "data packs" is the key point that Microsoft is missing in their understanding of the '808 Patent's teaching and claims over Eylon and Christiansen: the '808 Patent is not teaching about "pages". Pages are a concept of the storage system (or "memory hierarchy"): uniform block sizes onto which a file is mapped, containing arbitrary data, which is opaque to the storage system. Rather, the '808 Patent teaches "data packs", which map an offset and location to a corresponding "asset" which can use the contextual meaning of the asset and map it to alternative locations, including local and remote, as well as combine multiple "data packs" in a way that is sufficient to cover the block-sized region of the "page" of Eylon. Thus, while Eylon says its mapping structure is flexible, the teachings of Eylon focus on the flexibility of a simple block structure. There is nothing in Eylon that teaches a system aware of the internal structure of the application's variable sized objects. The "assets" or "data packs" of the '808 Patent do not conform

to the fixed location, fixed size units of Eylon. Further, the use of opaque, context-free, fixed sized storage units is consistent with the ordinary approach used by a POSITA within the computer storage field: storage relates to multiples of a fixed-size block. Traditional storage devices: disks, CD-ROMs, DVDs, SSDs, etc. are all organized around uniform “block” sized units and the deliberate decision not to understand there is structure of the data within those blocks.

150. Similarly, modern operating systems, including Windows, divide memory into “pages”. The concept of a page dates from the 1950s with the Atlas system. For example, in “Dynamic Storage Allocation in the Atlas Computer, Including an Automatic Use of a Backing Store” (Exhibit 2033), the author describes the meaning of the term “address” and “page”:

“The main memory registers are grouped in units of 512, but a careful distinction must be drawn between a 512-word block of information and a 512-word unit of memory; the latter is described as a page of core store or a sector of drum store. Thus, a block of information occupies either a page of core store or a sector of drum store. The programmer need not think of addresses in this way, but it is how they are treated by the machine.”

151. The use and meaning of these terms have not changed substantially since that time and is directly germane to the analysis of Eylon, Christiansen, and the ’808 Patent.

152. Microsoft’s references are all focused on the storage or memory perspective. Thus, Microsoft’s references consider some sort of uniform size: page, block, sector, streamlet. Microsoft’s references consider the data content of the uniform sized page, block, sector, or streamlet to be opaque and do not consider any structure that it might have. The ’808 Patent claims and teachings are from the perspective of considering that a file is a logical combination of “data packs” which can be identified using their offset and length and mapping it to an “asset” that has contextual meaning and location information, both local and remote, which can be used to reconstruct the full contents of the file in an efficient way that is not possible when using fixed

size opaque data blocks. Their understanding that files are a result of composition of such objects allows them to employ their understanding of the problem in a non-obvious fashion within the storage arena, which clearly distinguishes itself from the prior art. The '808 Patent could have used a storage-centric block model, in which case their teachings would not have been distinct from the prior art. However, it did not and instead the teachings of the '808 Patent provide an important insight: while traditional storage is agnostic to the internal structure of arbitrary file contents, when the contents are known that deeper understanding can be used to implement a highly efficient streaming on-demand system.

153. Thus, based upon my analysis, it is my opinion that Claim 8 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

154. Claim 9 of the '808 Patent reads as follows: "A streaming on demand system according to claim 1, said at least one address including and address for at least one local source for a first data pack, said local source being a source from the group consisting of a hard disk, an optical storage medium, a nonvolatile memory, a magnetic storage medium, and an address for at least one remote source for a 30 second data pack."

155. Claim 9 is a dependent claim that depends on Claim 1. I understand that Claim 9 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

156. Thus, based upon my analysis, it is my opinion that Claim 9 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

157. Claim 10 of the '808 Patent reads as follows: "A streaming on demand system according to claim 1, said at least one address including and address for at least one local source

for a first data pack, and an address for at least one remote source for a second data pack, said remote source being a source from the group consisting of a server and a peer computing device.”

158. Claim 10 is a dependent claim that depends on Claim 1. I understand that Claim 10 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

159. Thus, based upon my analysis, it is my opinion that Claim 10 of the ’808 Patent is not obvious using any combination of teachings of the cited prior art.

160. Claim 11 of the ’808 Patent reads as follows: “A streaming on demand system according to claim 1, said minifilter being configured to request to retrieve a required data pack from a first source corresponding to a first address in the table if the data pack is available from the first source, and to identify an alternative address in the table if the data pack is not available from the first source.”

161. Claim 11 is a dependent claim that depends on Claim 1. I understand that Claim 10 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

162. The specific analysis that Microsoft provides here regarding Christiansen appears to confuse the term “file” with the term “data pack”. Microsoft states: “Christiansen explains that, in response to an I/O request, the system makes a determination of whether the requested information is in a local cache (‘a first source’) ‘by obtaining the cache directory from the mapping table and attempting to open the object. EX1006, [0068]. In the combined system analyzed here, such a cached object would be a cached streamlet (“data pack”), and a Skilled Artisan would understand that an attempt to open such a cached object is a “request to retrieve a required data pack” because it is an attempt to access the contents of the streamlet from memory.”

163. The system of Christiansen is opening a *file*. Thus, Microsoft's interpretation of Christiansen only works if the file maps to the data pack. The nuances of offset and length that the '808 Patent teaches and claims is not taught in Christiansen because there is no reason for an offset and length because a reparse is performed against the entire file. The only way that I can reasonably envision Christiansen working in the same fashion is if data packs and files are in a one-to-one correspondence. However, a model in which data packs are one-to-one mappings of files is not consistent with the claims or teachings of the '808 Patent.

164. Microsoft also asserts that the access to the data pack can be "repared to the new name". Note that neither a file system filter nor minifilter can reparse a read or write I/O operation to a different name; the reparse operation to which Microsoft refers can only apply to the *open* of a given file. The use of a reparse only on file open makes no sense in the context of the '808 Patent's teachings where it teaches that the "data packs" needed to reconstruct a "data stream" may come from different locations in the same or different files on the same machine, or from different remote locations, or a mix of both. The teachings and claims of the '808 Patent are incompatible with the use of the REPARSE mechanism in Windows.

165. The '808 Patent teaches and claims the mapping of internal file structure ("data packs") to a distinct location, which need not be tied to the name space. For example, in the '808 Patent we could have two files that reference the same data pack, such as a program asset, and the mapping table would point both files to the relevant local data pack. Christiansen does not have data packs, nor the ability to identify multiple locations for a single file. The '808 Patent claims and teaches that each non-unique block or file could be mapped to the same data pack, which can be stored locally, or retrieved from one or more remote locations. Nothing in Christiansen teaches

complex mappings of the local cached file to the rich range of options described by the '808 Patent and thus Claim 11 of the '808 Patent cannot be derived from the teachings of Christiansen.

166. Further, Microsoft admits that Christiansen is file based, since Microsoft points out that Christiansen uses the STATUS\_REPARSE method to potentially redirect the open of the file to a different file within the cache. The reparse mechanism described by Christiansen is not sufficient to do the same thing for pieces of the file. Thus, Christiansen does not teach what the '808 Patent teaches and claims.

167. Thus, based upon my analysis, it is my opinion that Claim 11 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

168. Claim 12 of the '808 Patent reads as follows: “A streaming on demand system according to claim 1, said minifilter being configured, if the I/O request is a write operation, to mark an area as delivered and then write an address and size indicator corresponding to the write operation to the table.”

169. Claim 12 is a dependent claim that depends on Claim 1. I understand that Claim 12 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

170. As previously noted, neither Eylon nor Christiansen contemplate the claims of the '808 Patent: a system in which the offset and length of an I/O operation are mapped to a “data pack”, which captures the contextual meaning of the asset, and maintains information about locations from which the asset can be retrieved as well as how “data packs” can be used to cover a “data stream”, which might include an entire file and retrieve the relevant data. Instead, both Eylon and Christiansen teach a simple and direct mapping between remote and local storage in which a file consists of a set of fixed-size blocks that have a one-to-one mapping between the local

and remote file. The teachings of Eylon and Christiansen are clearly distinct from the teachings and claims of '808 Patent.

171. Microsoft asserts that a “Skilled Artisan” (POSITA) would have understood that it makes sense to use a previously fetched streamlet but neither Eylon nor Christiansen determines the data content of the storage blocks. Thus, duplicated blocks present in the remote computer’s file system of systems employing the teachings of Eylon and/or Christiansen will each be separately read to the corresponding location of the file of the client computer. While the problem of data deduplication can be resolved using techniques such as content addressing, in my experience, the concept of content mapping, versus logical location mapping is one that had not been exploited at the file system minifilter level for a class of applications, e.g., streaming on demand systems prior to the '808 Patent. One reason that the systems of Eylon and Christiansen will be inefficient for data deduplication is that a system of fixed size blocks will only find duplicate blocks with identical content. Thus, two blocks with the same assets in a different arrangement would not be considered duplicates in the systems of Eylon and Christiansen unlike systems embodying the teachings of the '808 Patent, where deduplication would be done based on assets. Eylon and Christiansen are constructing one-to-one mappings of remote to local logical file data, without any insight into the structure or contents of that data.

172. Microsoft further asserts that “[i]t would have been obvious, after such a write operation and invalidation operation, to immediately read that recently modified streamlet back into the local cache.” I respectfully disagree with Microsoft’s assertion – it is not obvious to me. It does not make sense to write a change to the remote server and then read the data back from the remote server because it is inefficient. Instead, it makes sense to simply write the data to local storage, rather than re-fetch it from the remote location.

173. Microsoft's assertion draws attention to another distinguishing feature of the claims of the '808 Patent: in a case where there are multiple instances of the same data pack, a modification to a single instance of that data pack can be overwritten without damaging the relationships to the other data packs. In the systems taught by Eylon and Christiansen, a write to the local system is sent to the remote system, which in turn means that every client computer system could eventually end up with a copy of the modified data that one client computer wrote back to the server computer. The '808 Patent neither teaches nor claims that data must be written back to a remote server. Instead, it teaches that the modification is made locally, for that specific instance of the data pack, and that information is used to update the mapping table. Selective update, whether based upon the asset, or based upon the locations from which that asset can be, is not possible in the systems of Eylon or Christiansen.

174. Microsoft also argues that, when Prahlad is combined with the teaching of Eylon and Christiansen, the claims of the '808 Patent become obvious. The limitations of the '808 Patent cannot be derived from the teachings of Prahlad when combined with Eylon and Christiansen because the '808 Patent teaches that a write to a data pack causes the data to be written to the local copy of the file. Only writing the data to the local system is distinct from the teachings of Prahlad, Eylon, and Christiansen, which teach that the changes to a file are written back to the original (remote) copy of that.

175. While the log of Prahlad is useful in batching together changes to be sent to one or more remote systems, it is not useful for determining updates that have been made locally. A log is useful when updating a remote system if delayed updates are allowed by that system. However, a log is not useful when determining changed regions on a local system because the logs must be processed locally to determine if there has been a change, which requires scanning the log files

each time an application performs a read I/O against any file. The log of Prahlaḍ is useful for updating the remote copy because the reference to the log is made once – when the updates are being applied to the remote system. The log of Prahlaḍ is not useful on the client because it requires a linear scan of the log file to determine the region that has changed. Other systems do a better job of supporting data replication of files in a file system than the system of Prahlaḍ, such as Cedar (Exhibit 2026) and Harp (Exhibit 2042). Thus, there is no reason to combine Prahlaḍ with Eylon or Christiansen. The '808 Patent does not teach or claim the update of the remote copy. Claim 12 of the '808 Patent claims the opposite: the data is written to a location on the local computer and the table is updated to reflect that the modified data pack is present on the local computer system.

176. The '808 Patent neither teaches nor claims the use of a log for updating either the local or remote copy of a file. Instead, the '808 Patent teaches and claims that a local modification of an asset is stored locally. Further, in the system described in the '808 patent, local modifications of the data can be performed even in the case where the underlying storage media of the current location is immutable, such as might be the case for data packs that are located on a CD-ROM, by writing the affected assets to mutable local storage and updating the asset table to point to the specific “data pack” location within the mutable local storage. The local update can be stored efficiently because the mapping table claimed and taught by the '808 Patent maps the modified “data pack” (asset) to one or more arbitrary locations. Thus, the location information for the modified data can be updated to point to an alternative location of the relevant “data pack”. Eylon and Christiansen only teach that changes are written back to the original file location.

177. A POSITA would not combine the teachings of Prahlad with Eylon, and/or Christiansen because there is no benefit to combining Prahlad with Eylon and/or Christiansen to track updates to locally stored files.

178. Microsoft asserted that a POSITA would write data back to the remote storage location and then read it back into the local location as described in ¶171. However, combining the teachings of Prahlad would require transmitting the log to the remote system. Prahlad teaches and claims quiescence of application programs and the insertion of consistency point markers into the log files. Thus, Prahlad does not process log files immediately, but rather only when the log indicates that the log is in a consistent state. Because Prahlad defers processing log files to consistent states when the application reads the modified data from the server, it must fetch that data from the remote server, but that may be delayed because the remote server has deferred updating the data until the log contains the necessary consistency marker. Combining Prahlad with Eylon and/or Christiansen leads to a system that could stall for an indeterminate amount of time, including the possibility of a “deadlock” situation where the application cannot quiesce until the read completes, but the read cannot complete until the application quiesces. Thus, the log system of Prahlad, combined with Eylon or Christiansen leads to complex behavior. Thus, combining Prahlad with Eylon and Christiansen would not be obvious to a POSITA.

179. Claim 12 of the '808 Patent does not require writing data back to the remote location, does not require delaying writes until the application quiesces, does not require delaying reads until modified data can be fetched from a remote server, and does not require scanning log files on the client system to satisfy a read. Thus, in my opinion, it is not obvious to combine the teachings of Prahlad, Eylon, and/or Christiansen, both for the reasons that I have previously noted (¶90) that Eylon and Christiansen implement systems that provide similar functionality and thus

do not make sense to combine, but also because the logging system of Prahlad is about the problem of creating change journals (or logs) that can be used for batching file updates into consistent sets as well as efficient transmission to one or more remote storage services. For these reasons, a POSTIA would not have found combining these three patents to be obvious.

180. Based upon my analysis, it is my opinion that Claim 12 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

181. Claim 13 of the '808 Patent reads as follows: "A streaming on demand system according to claim 1, said minifilter being configured, if the I/O request is a create operation, to attach extended data to a file stream, said extended data corresponding to the create operation."

182. Claim 13 is a dependent claim that depends on Claim 1. I understand that Claim 13 inherits each of the limitations of Claim 1. The analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen teach these limitations.

183. Christiansen does teach reparse points; claim 13 of the '808 Patent does not claim reparse points. Microsoft explains how to use per-file reparse points. While I could implement per-stream reparse points within the context of a file system minifilter and such an implementation could be combined with the teachings of the '808 Patent, it would not bridge the gap between Christiansen and the '808 Patent. It would not bridge the gap because a key teaching of the '808 Patent is that a "data stream" is made up of "data packs" and "data packs" are identified using the offset and length relative to a particular "data stream" and then mapped to the contextual meaning of the data pack via a lookup table. An embodiment of the '808 Patent could use the NTFS reparse point facility as a trigger for creating and managing the mapping table limitation of the '808 Patent. The '808 Patent does not explicitly claim to use reparse points for triggering a distinguished status condition, but it is one possible example of per-stream context distinct from the per-stream context

that is already provided to file systems, via the FileObject data structure, and file system filter drivers, via the FsRtl functions exported from the Windows kernel.

184. One reason to use an embodiment that employs reparse points is that because, unlike the file system minifilters that Microsoft provides to their customers, a file system minifilter developed by a third party could be removed from the system. Using a custom reparse point value, ordinary applications would not be able to access these files, ensuring that such applications do not obtain erroneous information or inadvertently modify the contents of the file. One reason to eschew such an embodiment that employs reparse points is because most file systems do not natively support reparse points, though both the NTFS and ReFS file systems on Windows do support them.

185. Microsoft conflates Christiansen's use of a reparse point (Christiansen starting at paragraph 0069, notably paragraph 81, and Figure 9) with the technique of the '808 Patent that maps each distinct data pack using a table to a separate location for retrieval. In Christiansen, the reparse point is simply used to redirect the open of a file from one location in the namespace. The reparse points of Christiansen are consistent with the "links" that are taught by Daley in the 1965 Multics I/O paper (Exhibit 2021). It does not require Christiansen to teach it. However, what is different about reparse points in Windows is that reparse points can also be used to "draw attention" to the fact that a given file requires processing by a particular file system filter driver. In that case, the file inside a file system such as NTFS that supports reparse points will raise an exception condition indicating that the file should not be directly accessed by an application. File system minifilters can use that technique to indicate additional processing of the file is required. The reparse point interception technique is one that Microsoft has been teaching since Windows NT 3.1 (1993), which is prior to the time of Eylon.

186. Microsoft also argues that the combination of Thind, Eylon, and Christiansen make the claims of the '808 Patent obvious. I note that Thind's claim on associating context with a stream context is not, by itself, novel or interesting. I was teaching file system filter driver developers how to add the equivalent of file, stream, and stream handle contexts – all concepts presently included in the Windows filter manager, at least as early as 1996. The use of file, stream, and stream handle contexts was known at the time because a Windows file system, such as NTFS, had its own mechanism for storing exactly this context via the FileObject passed to the file system. How Windows file systems associated their context with an open instance of a file (a “stream handle context”) or the specific instance of a file stream (a “stream context”) or context associated with *all* streams of a given file (a “file context”) were certainly well-understood by 1997 (Exhibit 2017). Accordingly, I and others were able to build exactly these types of context tracking mechanisms. Microsoft controls the Windows operating system and decided to simplify the management of file, stream, and stream handle contexts by providing a new table of related contexts and modified the Windows operating system as well as Windows file systems to support the new context model; in fact, I assisted in finding a flaw in the implementation of these new contexts in the Windows 2000 version of the new context implementation that caused a failure for some third party network file systems for the Windows 2000 platform.

187. Thus, while I agree that the use of contexts was well known by the time of Thind, the combination of Eylon and Christiansen do not teach the use of maintaining a table of variable sized data packs with at least one local or remote location, reachable using the stream contexts of the Windows filter manager. I have previously noted that combining Eylon and Christiansen does not make sense because these two references teach different ways to accomplish a similar purpose (¶90). Combining Christiansen with Thind does not yield anything additional, since Christiansen

is already describing a file system filter driver, so there would be no obvious motivation to combine them; many legacy file system filter driver products continued to exist for a decade after Microsoft added filter manager to the Windows operating system. Indeed, in 2015 I worked on converting a legacy filter driver for a company that had sold their SQL server backup product to Microsoft. Part of that product was a working legacy file system filter driver that I replaced with a file system minifilter driver. There was no technical reason that a file system minifilter was needed; it was a commercial requirement that Microsoft imposed as a condition of buying the company's solution. Thus, there is no obvious reason to combine these references, even if Christiansen, Eylon, and Thind did combine to teach the claims of the '808 Patent.

188. No combination of Eylon, Christiansen and Thind alters my analysis of Claim 13 of the '808 Patent: Thind does not bridge the gap between Eylon and/or Christiansen and the '808 Patent.

189. Based upon my analysis, it is my opinion that Claim 13 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

190. Claim 14 of the '808 Patent reads as follows: "A streaming on demand method comprising installing an on-demand requester object on a computing device having an I/O manager, said on-demand requester object being configured to receive I/O requests from the I/O manager on behalf of an application for which data is available in data packs for streaming delivery, said on-demand requester object being responsive to application I/O requests, said application I/O requests corresponding to sequential and nonsequential data packs, said data packs being portions of a complete data stream, said I/O requests determining an order in which data packs are sought for the application, said on demand requester object comprising a minifilter associated with a filter manager in an I/O stack, said filter manager being configured to call the

minifilter, said minifilter having a preoperation callback routine for the I/O request registered with the filter manager, and said minifilter receiving each I/O request from the I/O manager, said minifilter referencing a table that includes names and paths with at least one offset address and length where each data pack required to fulfill each I/O request is located, said minifilter determining if the data pack required to fulfill each I/O request been streamed to the system, and said application using the data packs that fulfill each I/O request when the data packs are available on the computing device.”

191. Claim 1 is nearly identical to Claim 14 but Claim 14 includes the further limitation of the step of installing into an existing system. Thus, the analysis of Claim 1 (¶¶74-91) explains why neither Eylon nor Christiansen, nor a combination of Eylon and Christiansen teach the idea of installing or upgrading a system that supports streaming on-demand download of applications in which the combination of file, offset, and length are mapped using a table to one or more “data packs” which capture the context of the individual data packs, corresponding to assets with meaning to the application, and maintaining a list of locations from which the given asset can be obtained. Thus, an embodiment of the ’808 Patent that adds or upgrades the functionality that is taught by the ’808 Patent would similarly not be taught by Eylon or Christiansen.

192. Based upon my analysis, it is my opinion that Claim 14 of the ’808 Patent is not obvious using any combination of teachings of the cited prior art.

193. Claim 15 of the ’808 Patent reads as follows: “A streaming on demand method according to claim 14 further comprising attaching said minifilter to the I/O stack by registering with a filter manager in the I/O stack for I/O requests for the application for which data is available in data packs for streaming delivery.”

194. Claim 15 is a dependent claim that depends on Claim 14. I understand that Claim 15 inherits each of the limitations of Claim 14. The analysis of Claim 14 (¶190) explains why neither Eylon nor Christiansen teach these limitations.

195. Based upon my analysis, it is my opinion that Claim 15 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

196. Claim 16 of the '808 Patent reads as follows: "A streaming on demand method according to claim 14 further comprising, said minifilter passing on a read call if a required data pack is available locally and if all data in the required data pack has been received."

197. Claim 16 is a dependent claim that depends on Claim 14. I understand that Claim 16 inherits each of the limitations of Claim 14. The analysis of Claim 14 (¶190) explains why neither Eylon nor Christiansen teach these limitations.

198. In addition, the analysis that I included for Claim 5 (¶127) similarly applies to Claim 16.

199. Based upon my analysis, it is my opinion that Claim 16 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

200. Claim 17 of the '808 Patent reads as follows: "A streaming on demand method according to claim 14 further comprising, said minifilter withholding a read call if a required data pack is not available locally."

201. The analysis of Claim 14 (¶190) explains why neither Eylon nor Christiansen teach these limitations.

202. In addition, the analysis that I provided for Claim 6 (¶133-134) applies to Claim 17.

203. Based upon my analysis, it is my opinion that Claim 17 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

204. Claim 18 of the '808 Patent reads as follows: "A streaming on demand method according to claim 14 further comprising, said minifilter withholding a read call if all data in the required data pack has not been received."

205. The analysis of Claim 14 (§190) explains why neither Eylon nor Christiansen teach these limitations.

206. In addition, the analysis that I provided for Claim 7 (§138) applies to Claim 18 as well.

207. Based upon my analysis, it is my opinion that Claim 17 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

208. Claim 19 of the '808 Patent reads as follows: "A streaming on demand method according to claim 14, said at least one address including and address for at least one local source for a first data pack, said local source being a source from the group consisting of a hard disk, an optical storage medium, a nonvolatile memory, a magnetic storage medium, and an address for at least one remote source for a second data pack: said remote source being a source from the group consisting of a server and a peer computing device."

209. The analysis of Claim 14 (§190) explains why neither Eylon nor Christiansen teach these limitations.

210. In addition, the analysis that I provided for Claim 11 (§161-**Error! Reference source not found.**) applies to Claim 19 as well. Thus, in my opinion, the claims of the '808 Patent are not obvious using any combination of the teachings of the cited prior art.

211. Claim 20 of the '808 Patent reads as follows: "A streaming on demand method according to claim 14, said minifilter requesting to retrieve a required data pack from a first source

corresponding to a first address in the table if the data pack is available from the first source, and identifying an alternative address in the table if the data pack is not available from the first source.”

212. The analysis of Claim 14 (¶190) explains why neither Eylon nor Christiansen teach these limitations.

213. In addition, the language of Claim 20 claim underscores the teaching that the data packs of the '808 Patent need not be stored in any correspondence with their location inside a specific file. The terms here do not indicate that the addresses are local or remote; a reasonable interpretation of the text is that the addresses can be one or more of local and remote addresses. Since the address is for the data pack (“asset”) and not the file, the mapping table associated the given data pack within some file with the location of that data pack, possibly in the same file on the local computer, possibly in an unrelated file on the local computer, or in some remote location.

214. Based upon my analysis, it is my opinion that Claim 17 of the '808 Patent is not obvious using any combination of teachings of the cited prior art.

DECLARATION OF WILLIAM ANTHONY MASON  
Case IPR2021-00015  
U.S. Patent No. 8,380,808

I, William Anthony Mason, do hereby declare and state that all statements made herein of my own knowledge are true and that all statements made on information and belief are believed to be true; and further that these statements were made with the knowledge that willful false statements and the like so made are punishable by fine or imprisonment, or both, under Section 1001 of Title 18 of the United States Code.

Dated: February 17, 2021

  
William Anthony Mason

DECLARATION OF WILLIAM ANTHONY MASON

Case IPR2021-00015

U.S. Patent No. 8,380,808

**CERTIFICATE OF SERVICE**

I hereby certify that on February 17, 2021, a copy of this **DECLARATION OF WILLIAM ANTHONY MASON**, including all attachments, appendices, and exhibits thereto, has been served in its entirety on attorneys for petitioner, via electronic mail, at the following:

Joseph A. Micallef, Reg. No. 39,772  
Scott M. Border, Reg. No. 77,744  
**SIDLEY AUSTIN LLP**  
1501 K Street, N.W.  
Washington, DC 20005  
Tel: (202) 736-8000  
Email: [jmicallef@sidley.com](mailto:jmicallef@sidley.com)  
[sborder@sidley.com](mailto:sborder@sidley.com)  
[iprnotices@sidley.com](mailto:iprnotices@sidley.com)

***Attorneys for Petitioner***

Nathan A. Greenblatt, Reg. No. 61,265  
**SIDLEY AUSTIN LLP**  
1001 Page Mill Road, Building 1  
Palo Alto, CA 94304  
Tel: (650) 565-7000  
Email: [ngreenblatt@sidley.com](mailto:ngreenblatt@sidley.com)  
[iprnotices@sidley.com](mailto:iprnotices@sidley.com)

***Attorneys for Petitioner***

Dated: February 17, 2021

Respectfully Submitted,

s/ Mark Lawrence Lorbiecki  
Mark Lawrence Lorbiecki, Reg. No. 45,643  
**WILLIAMS, KASTNER & GIBBS PLLC**  
601 Union Street, Suite 4100  
Seattle, WA 98101-2380  
Tel: (206) 628-6600  
Fax: (206) 628-6611  
Email: [mlorbiecki@williamskastner.com](mailto:mlorbiecki@williamskastner.com)

***Attorneys for Patent Owner***